RX ファミリ

RSCI 簡易 I²C モジュール Firmware Integration Technology

要旨

本アプリケーションノートは、Firmware Integration Technology (FIT)を使用した簡易 I²C モジュールについて説明します。本モジュールはシリアルインタフェース(RSCI)を使用して、デバイス間で通信を行います。以降、本モジュールを RSCI (簡易 I²C モード) FIT モジュールと称します。

対象デバイス

以下は、この API によってサポートできるデバイスの一覧です。

- RX260、RX261 グループ
- RX26T グループ
- RX660 グループ
- RX671 グループ

本アプリケーションノートを他のマイコンへ適用する場合、そのマイコンの仕様にあわせて変更し、十分評価してください。

対象コンパイラ

- Renesas Electronics C/C++ Compiler Package for RX Family
- GCC for Renesas RX
- IAR C/C++ Compiler for Renesas RX

各コンパイラの動作確認内容については「5.3 動作確認環境詳細」を参照してください。

目次

1. 概要 4
1.1 RSCI (簡易 I²C モード) FIT モジュールとは 4
1.2 API の概要 5
1.3 RSCI (簡易 I²C モード) FIT モジュールの概要 6
1.3.1 RSCI (簡易 I²C モード) FIT モジュールの仕様 6
1.3.2 マスタ送信の処理 7
1.3.3 マスタ受信の処理 11
1.3.4 状態遷移図 14
1.3.5 状態遷移時の各フラグ 15
1.4 RSCI (簡易 I2C モード) FIT モジュールの使用方法 15
1.4.1 RSCI (簡易 I2C モード) FIT モジュールを C++プロジェクト内で使用する方法 15

2. API 情報 16
2.1 ハードウェアの要求 16
2.2 ソフトウェアの要求 16
2.3 サポートされているツールチェーン 16
2.4 使用する割り込みベクタ 17
2.5 ヘッダファイル 18
2.6 整数型 18
2.7 コンパイル時の設定 19
2.8 コードサイズ 22
2.9 引数 23
2.10 戻り値 23
2.11 モジュールの追加方法 24
2.12 for 文、while 文、do while 文について 25

3. API 関数 26
R_RSCI_IIC_Open() 26
R_RSCI_IIC_MasterSend() 28
R_RSCI_IIC_MasterReceive() 34
R_RSCI_IIC_Close() 37
R_RSCI_IIC_GetStatus() 38
R_RSCI_IIC_Control() 40
R_RSCI_IIC_GetVersion() 42

4. 端子設定 43

5. 付録 44
5.1 通信の実現方法 44
5.1.1 API 動作の状態 44
5.1.2 API 動作中のイベント 44
5.1.3 プロトコル状態遷移 45
5.1.4 プロトコル状態遷移表 49
5.1.5 プロトコル状態遷移登録関数 49
5.1.6 状態遷移時の各フラグの状態 50
5.2 割り込み発生タイミング 52
5.2.1 マスタ送信 52
5.2.2 マスタ受信 53
5.2.3 マスタ送受信 54
5.3 動作確認環境詳細 55
5.4 トラブルシューティング 56

6. サンプルコード 57
6.1 1 つのチャネルで 1 つのスレーブデバイスに連続アクセスする場合の例 57
6.2 1 つのチャネルで 2 つのスレーブデバイスにアクセスする場合の例 61
6.3 2 つのチャネルで 2 つのスレーブデバイスにアクセスする場合の例 66

7. 参考ドキュメント 72

テクニカルアップデートの対応について 73

改訂記録 74

1. 概要

RSCI (簡易 I²C モード) FIT モジュールは、RSCI を使用し、マスタデバイスがスレーブデバイスと送受信を行うための手段を提供します。RSCI の簡易 I²C モードは、NXP 社が提唱する I²C バス(Inter-IC-Bus)インタフェース方式のシングルマスタモードに準拠しています。以下に本モジュールがサポートしている機能を列挙します。

- シングルマスタモードのみに対応(スレーブ送信、スレーブ受信には非対応)
- コンディション波形の生成
- 通信モードはスタンダードモードとファストモードに対応し、最大転送速度は 384kbps

制限事項
- DMAC、DTC と組み合わせて使用することはできません。
- 10 ビットアドレスの送信には対応していません。
- 多重割り込みには対応していません。
- コールバック関数内では R_RSCI_IIC_GetStatus 関数以外の API 関数のコールは禁止です。
- 割り込みを使用するため、I フラグは“1”で使用してください。
- RSCI(簡易 I²C モード) FIT モジュールと RSCI モジュール Firmware Integration Technology(R01AN5759)を組み合わせて使用するとき、同じチャネルを同時に使用することはできません。

1.1 RSCI (簡易 I²C モード) FIT モジュールとは

本モジュールは API として、プロジェクトに組み込んで使用します。本モジュールの組み込み方については、2.11 を参照してください。

1.2 API の概要

表 1.1 に本モジュールに含まれる API 関数を示します。

表 1.1 API 関数一覧

関数	関数説明
R_RSCI_IIC_Open()	この関数は RSCI(簡易 I²C モード) FIT モジュールを初期化する関数です。この関数は他の API 関数を使用する前に実行される必要があります。
R_RSCI_IIC_MasterSend()	マスタ送信を開始します。 引数に合わせてデータ送信パターンを変更します。 ストップコンディション生成まで一括で実施します。
R_RSCI_IIC_MasterReceive()	マスタ受信を開始します。 引数に合わせてデータ受信パターンを変更します。 ストップコンディション生成まで一括で実施します。
R_RSCI_IIC_Close()	簡易 I²C の通信を終了し、使用していた RSCI の対象チャネルを解放します。
R_RSCI_IIC_GetStatus()	本モジュールの状態を返します。
R_RSCI_IIC_Control()	各コンディション出力、NACK 出力、SSCL クロックのワンショット出力、および本モジュールリセットを行う関数です。主に通信エラー時に使用してください。
R_RSCI_IIC_GetVersion()	本モジュールのバージョンを返します。

1.3 RSCI (簡易 I²C モード) FIT モジュールの概要

1.3.1 RSCI (簡易 I²C モード) FIT モジュールの仕様

1) 本モジュールでは、マスタ送信、マスタ受信 をサポートします。
　－マスタ送信では、4 種類の送信パターンが設定可能です。マスタ送信の詳細は「1.3.2 マスタ送信の処理」に示します。
　－マスタ受信では、マスタ受信とマスタ送受信の 2 種類の受信パターンが設定可能です。
　　マスタ受信の詳細は「1.3.3 マスタ受信の処理」に示します。

2) 割り込みは、スタートコンディション生成、スレーブアドレス送信、データ受信、ストップコンディション生成のいずれかの処理が完了すると発生します。RSCI(簡易 I²C モード)の割り込み内で本モジュールの通信制御関数を呼び出し、処理を進めます。

3) 本モジュールは、複数のチャネルを制御することができます。また複数チャネルを持つデバイスでは複数チャネルを使用して同時に通信することができます。

4) 1 つのチャネル・バス上の複数のスレーブデバイスを制御できます。
　ただし通信中(スタートコンディション生成から、ストップコンディション生成完了までの期間)は、そのデバイス以外の通信はできません。図 1.1 に複数スレーブデバイスの制御例を示します。

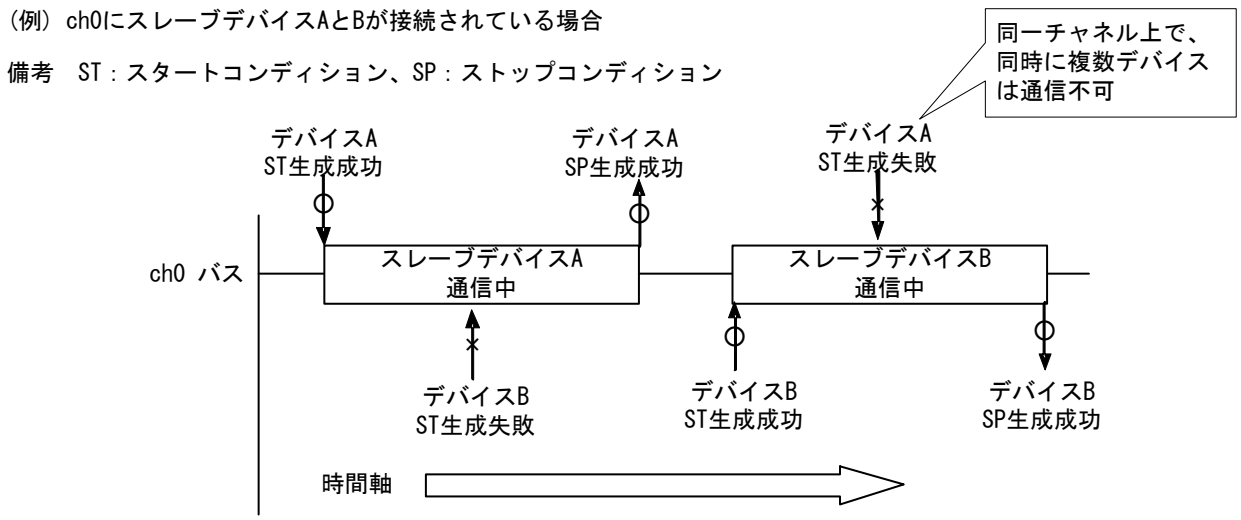


図 1.1 複数スレーブデバイス制御例

1.3.2　マスタ送信の処理

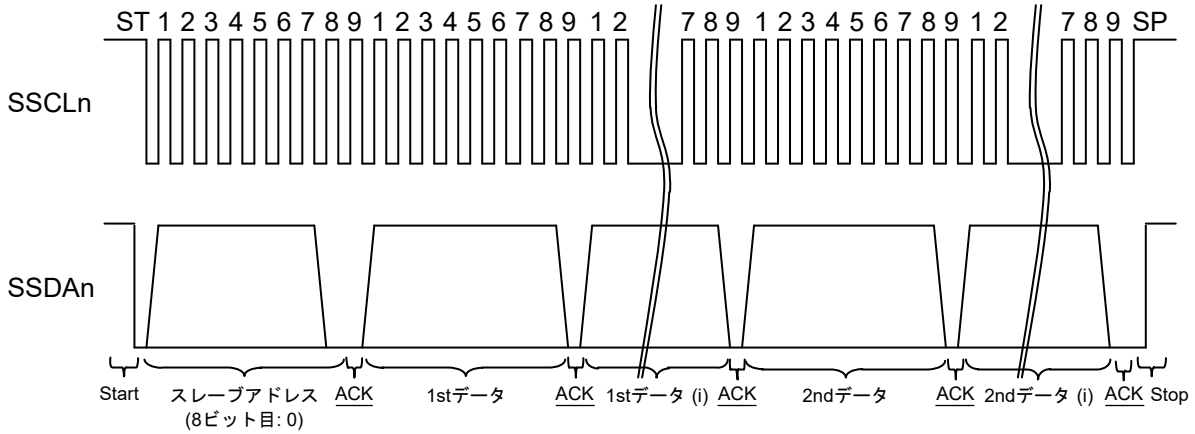
マスタデバイス (RX MCU) からスレーブデバイスへデータを送信します。

本モジュールでは、マスタ送信は 4 種類の波形を生成できます。マスタ送信の際の引数とする I²C 通信情報構造体の設定値によってパターンを選択します。I²C 通信情報構造体の詳細は、「2.9 引数」を参照してください。図 1.2～図 1.5 に 4 種類の送信パターンを示します。

(1) パターン 1

マスタデバイス(RX MCU)からスレーブデバイスへデータを送信します。

初めにスタートコンディション(ST)を生成し、次にスレーブデバイスのアドレスを送信します。このとき、8 ビット目は転送方向指定ビットになりますので、データ送信時には“0”(Write)を送信します。次に 1st データを送信します。1st データとは、データ送信を行う前に、事前に送信したいデータがある場合に使用します。例えばスレーブデバイスが EEPROM の場合、EEPROM 内部のアドレスを送信することができます。次に 2nd データを送信します。2nd データがスレーブデバイスへ書き込むデータになります。データ送信を開始し、全データの送信が完了すると、ストップコンディション(SP)を生成してバスを解放します。



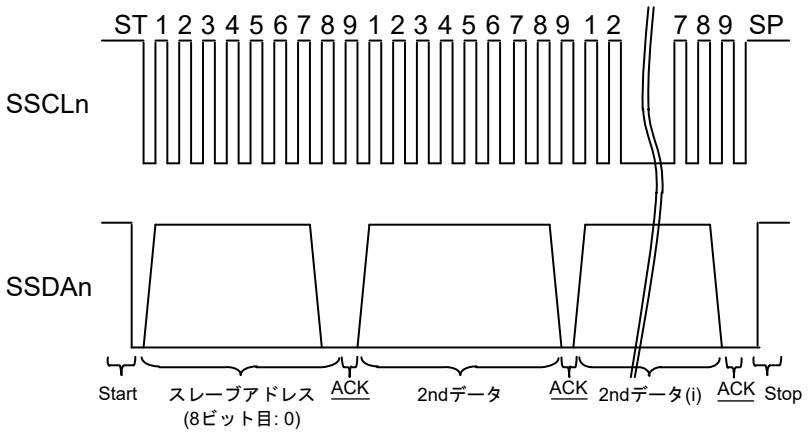
凡例:
n　：チャネル番号
ST：スタートコンディション生成
SP：ストップコンディション生成
ACK：Acknowledge“0”
※ 下線のある信号は、スレーブからマスタへのデータ送信です

図 1.2 マスタ送信(パターン 1)信号図

(2) パターン 2

マスタ(RX MCU)からスレーブデバイスへデータを送信します。ただし、1st データを設定していない場合、1st データは送信しません。

スタートコンディション(ST)を生成からスレーブデバイスのアドレスを送信まではパターン 1 と同様に動作します。次に 1st データを送信せず、2nd データを送信します。全データの送信が完了すると、ストップコンディション(SP)を生成してバスを解放します。



凡例:
n　：チャネル番号
ST：スタートコンディション生成
SP：ストップコンディション生成
ACK：Acknowledge“0”
※ 下線のある信号は、スレーブからマスタへのデータ送信です

図 1.3 マスタ送信(パターン 2) 信号図

(3) パターン 3

スタートコンディション(ST)を生成から、スレーブアドレス送信まではパターン 1 と同様に動作します。スレーブアドレス送信後、1st データ/2nd データを設定していない場合、データ送信は行わず、ストップコンディション(SP)を生成してバスを解放します。

接続されているデバイスを検索する場合や、EEPROM 書き換え状態を確認する Acknowledge Polling を行う際に有効な処理です。

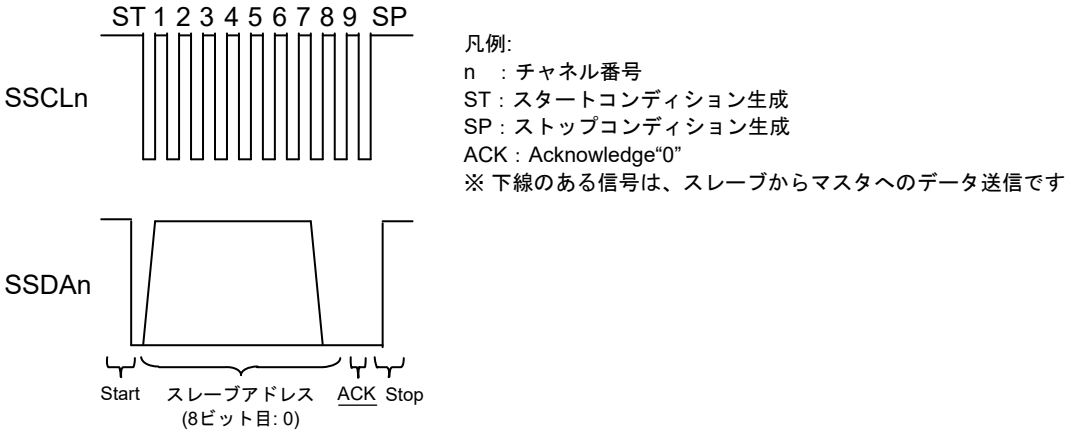


図 1.4 マスタ送信(パターン 3)信号図

(4) パターン 4

スタートコンディション(ST)を生成後、スレーブアドレスと 1st データ/2nd データを設定していない場合、スレーブアドレス送信とデータの送信は行わず、ストップコンディション(SP)を生成してバスを解放します。バス解放のみを行いたい場合に有効な処理です。

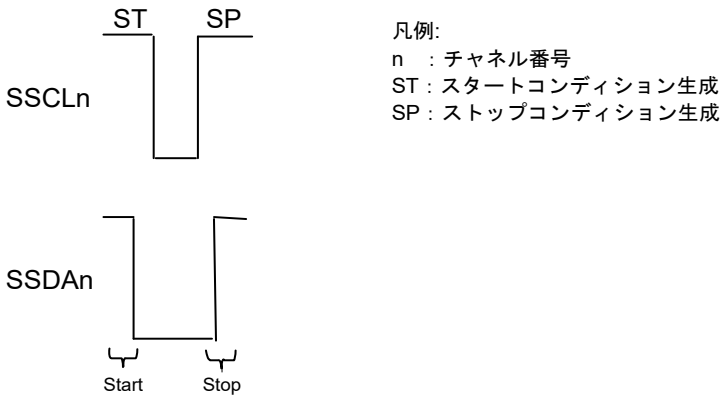


図 1.5 マスタ送信(パターン 4) 信号図

図 1.6 にマスタ送信を行う際の手順を示します。コールバック関数は、ストップコンディション生成後に呼ばれます。I²C 通信情報構造体メンバの CallBackFunc に関数名を指定してください。

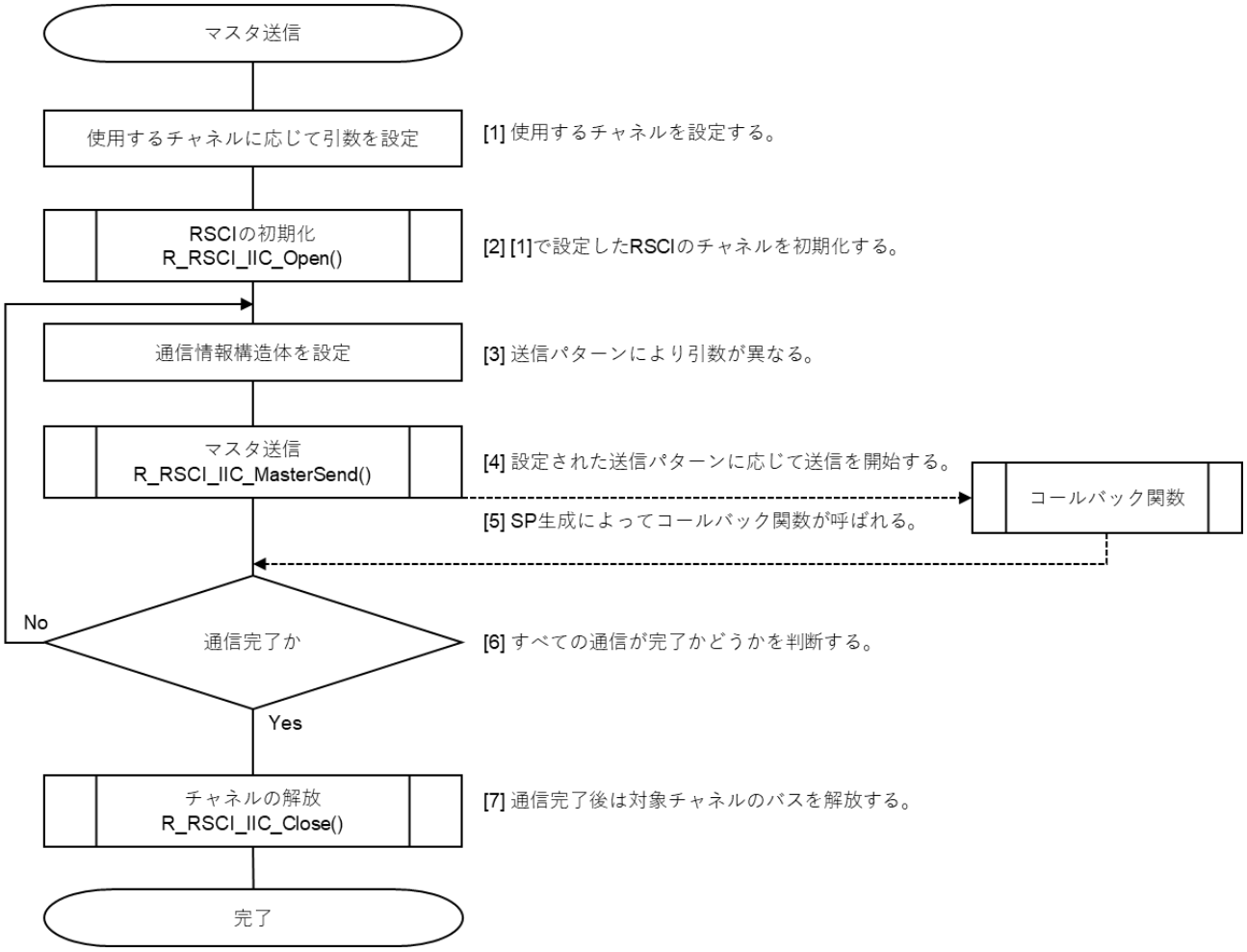


図 1.6 マスタ送信の処理例

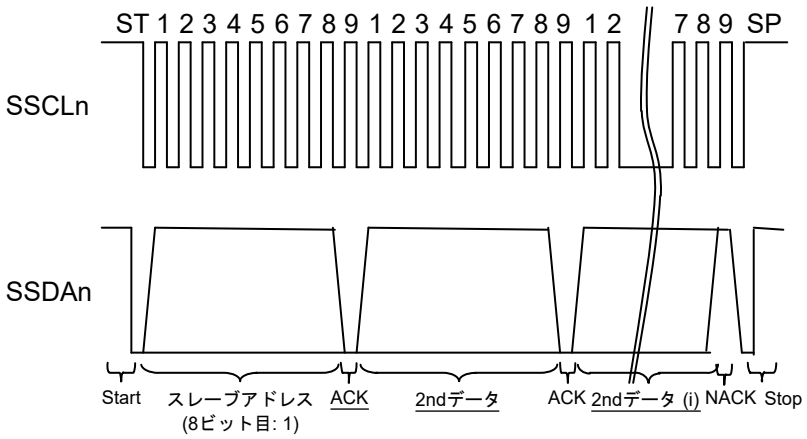
1.3.3 マスタ受信の処理

マスタデバイス(RX MCU)はスレーブデバイスからデータを受信します。本モジュールでは、マスタ受信とマスタ送受信に対応しています。マスタ受信の際の引数とする I²C 通信情報構造体の設定値によってパターンを選択します。I²C 通信情報構造体の詳細は、「2.9 引数」を参照してください。図 1.7～図 1.8 に受信パターンを示します。

(1) マスタ受信

マスタ(RX MCU)はスレーブデバイスからデータを受信します。

初めにスタートコンディション(ST)を生成し、次にスレーブデバイスのアドレスを送信します。このとき、8 ビット目は転送方向指定ビットになりますので、データ受信時には“1”(Read)を送信します。次にデータ受信を開始します。受信中は、1 バイト受信するごとに ACK を送信しますが、最終データ時のみ NACK を送信し、スレーブデバイスへ受信処理が完了したことを通知します。全データの受信が完了すると、ストップコンディション(SP)を生成してバスを解放します。



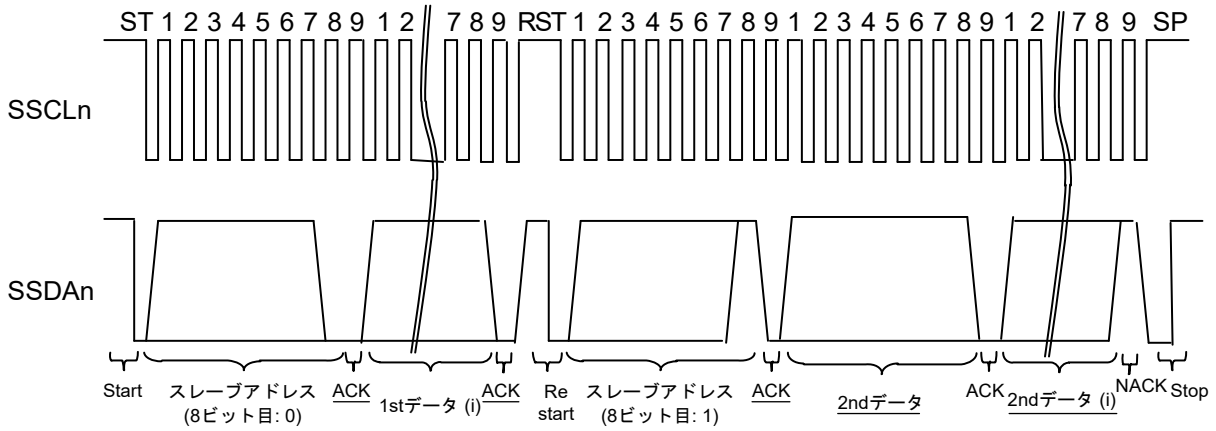
凡例:
n ：チャネル番号
ST: スタートコンディション生成　NACK: Acknowledge “1”
SP: ストップコンディション生成　ACK: Acknowledge “0”
※ 下線のある信号は、スレーブからマスタへのデータ送信です

図 1.7 マスタ受信　信号図

(2) マスタ送受信

マスタ(RX MCU)からスレーブデバイスへデータを送信します(マスタ送信)。送信完了後、リスタートコンディションを生成し、転送方向を“1”(Read)に変更して、マスタはスレーブデバイスからデータを受信します(マスタ受信)。

初めにスタートコンディション(ST)を生成し、次にスレーブデバイスのアドレスを送信します。このとき、8 ビット目は転送方向指定ビットになりますので、データ送信時には“0”(Write)を送信します。次に 1st データを送信します。データの送信が完了すると、リスタートコンディション(RST)を生成し、スレーブアドレスを送信します。このとき、転送方向指定ビットは“1”(Read)を送信します。次にデータ受信を開始します。受信中は、1 バイト受信するごとに ACK を送信しますが、最終データ時のみ NACK を送信し、スレーブデバイスへ受信処理が完了したことを通知します。全データの受信が完了すると、ストップコンディション(SP)を生成してバスを解放します。



凡例:
n　: チャネル番号
ST: スタートコンディション生成　　NACK: Acknowledge “1”
SP: ストップコンディション生成　　ACK: Acknowledge “0”
RST: リスタートコンディション生成
※ 下線のある信号は、スレーブからマスタへのデータ送信です

図 1.8 マスタ送受信　信号図

図 1.9 にマスタ受信を行う際の手順を示します。コールバック関数は、ストップコンディション生成後に呼ばれます。I²C 通信情報構造体メンバの CallBackFunc に関数名を指定してください。

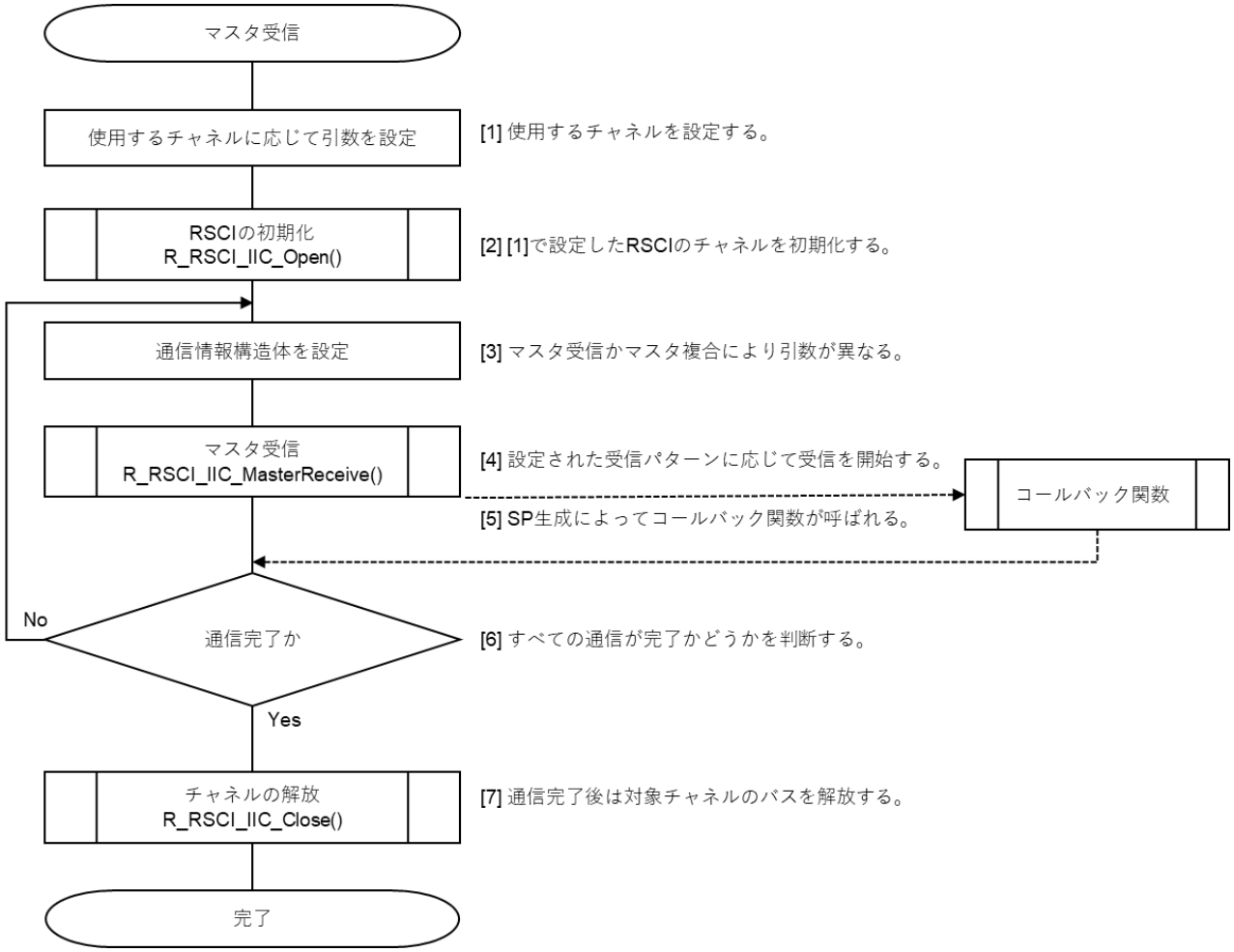


図 1.9 マスタ受信の処理例

1.3.4 状態遷移図

本モジュールを使用する場合、未初期化状態、アイドル状態、通信中のいずれかの状態になります。
図 1.10 に、状態遷移図を示します。

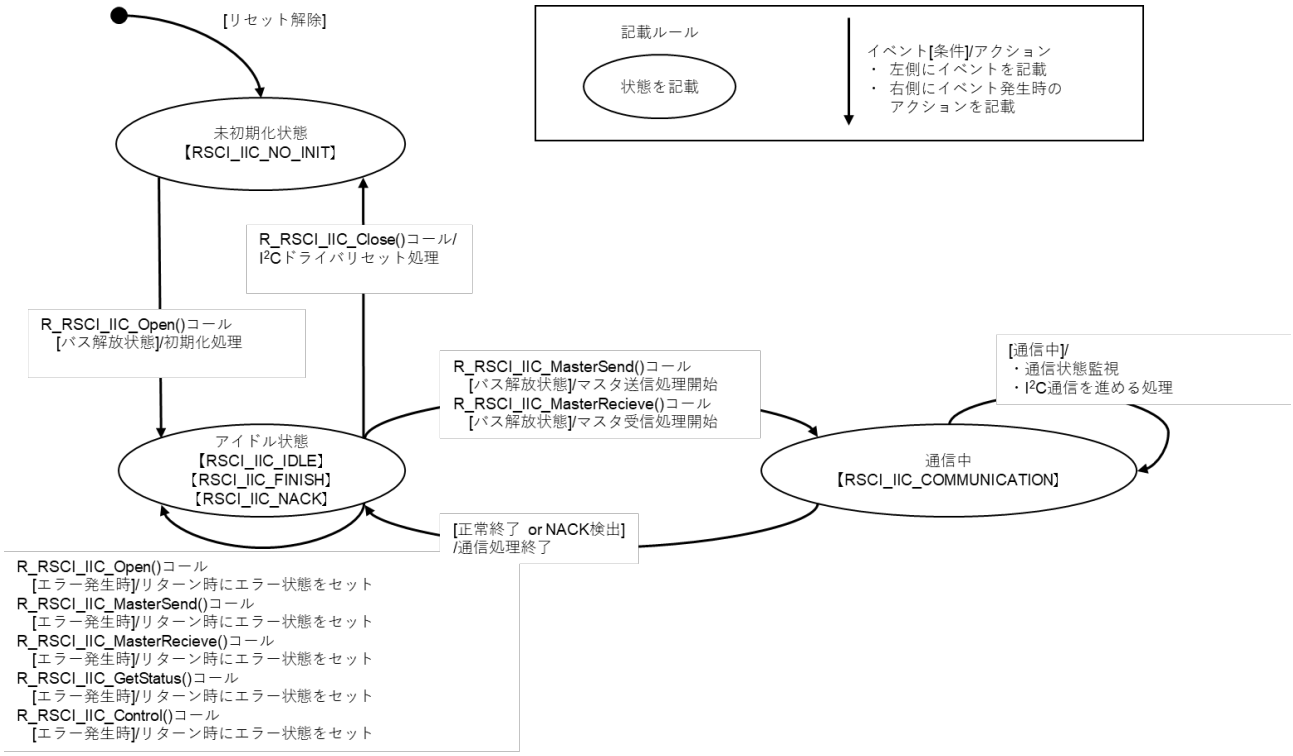


図 1.10 状態遷移図

1.3.5 状態遷移時の各フラグ

I²C 通信情報構造体メンバにはデバイス状態フラグ(dev_sts)があります。デバイス状態フラグには、そのデバイスの通信状態が格納されます。またこのフラグにより、同一チャネル上の複数のスレーブデバイスの制御を行うことができます。表 1.3 に状態遷移時のデバイス状態フラグの一覧を示します。

表 1.3 状態遷移時のデバイス状態フラグの一覧

状態	デバイス状態フラグ(dev_sts)
未初期化状態	RSCI_IIC_NO_INIT
アイドル状態	RSCI_IIC_IDLE RSCI_IIC_FINISH RSCI_IIC_NACK
通信中(マスタ送信)	RSCI_IIC_COMMUNICATION
通信中(マスタ受信)	RSCI_IIC_COMMUNICATION
通信中(マスタ送受信)	RSCI_IIC_COMMUNICATION
エラー	RSCI_IIC_ERROR

1.4 RSCI (簡易 I2C モード) FIT モジュールの使用方法

1.4.1 RSCI (簡易 I2C モード) FIT モジュールを C++プロジェクト内で使用する方法

C++プロジェクトでは、RSCI (簡易 I2C モード) FIT モジュールのインタフェースヘッダファイルを extern “C”の宣言に追加してください。

```
extern “C”
{
    #include “r_smc_entry.h”
    #include “r_rsci_iic_rx_if.h”
}
```

2. API 情報

本モジュールの API はルネサスの API の命名基準に従っています。

2.1 ハードウェアの要求

ご使用になる MCU が以下の機能をサポートしている必要があります。

- RSCI

2.2 ソフトウェアの要求

このドライバは以下のパッケージに依存しています。

- ボードサポートパッケージモジュール (r_bsp) Rev.7.70 以上

2.3 サポートされているツールチェーン

本 FIT モジュールは「5.3 動作確認環境詳細」に示すツールチェーンで動作確認を行っています。

2.4 使用する割り込みベクタ

R_RSCI_IIC_MasterSend 関数、R_RSCI_IIC_MasterReceive 関数を実行したとき、引数のパラメータで指定したチャネル番号のチャネルに対応した TXI 割り込み、TEI 割り込みが有効になります。

表 2.1 に RSCI(簡易 I²C モード) FIT モジュールが使用する割り込みベクタを示します。

表 2.1 使用する割り込みベクター覧

デバイス	割り込みベクタ
RX260 RX261	TXI 割り込み[チャネル 0]（ベクタ番号: 216） TEI 割り込み[チャネル 0]（ベクタ番号: 217） TXI 割り込み[チャネル 8]（ベクタ番号: 232） TEI 割り込み[チャネル 8]（ベクタ番号: 233） TXI 割り込み[チャネル 9]（ベクタ番号: 236） TEI 割り込み[チャネル 9]（ベクタ番号: 237）
RX26T	TXI 割り込み[チャネル 8]（ベクタ番号: 101） TXI 割り込み[チャネル 9]（ベクタ番号: 103） TXI 割り込み[チャネル 11]（ベクタ番号: 115） GROUPBL1 割り込み（ベクタ番号: 111） • TEI 割り込み[チャネル 8]（グループ割り込み要因番号: 24） • TEI 割り込み[チャネル 9]（グループ割り込み要因番号: 26） GROUPAL0 割り込み（ベクタ番号: 112） • TEI 割り込み[チャネル 11]（グループ割り込み要因番号: 12）
RX660 RX671	TXI 割り込み[チャネル 10]（ベクタ番号: 33） TXI 割り込み[チャネル 11]（ベクタ番号: 43） GROUPAL0 割り込み（ベクタ番号: 112） • TEI 割り込み[チャネル 10]（グループ割り込み要因番号: 24） • TEI 割り込み[チャネル 11]（グループ割り込み要因番号: 27）

2.5 ヘッダファイル

すべての API 呼び出しと使用されるインタフェース定義は r_rsci_iic_rx_if.h に記載しています。

2.6 整数型

このプロジェクトは ANSI C99 を使用しています。これらの型は stdint.h で定義されています。

2.7 コンパイル時の設定

本モジュールのコンフィギュレーションオプションの設定は、r_rsci_iic_rx_config.h、r_rsci_iic_rx_pin_config.h で行います。オプション名および設定値に関する説明を、下表に示します。

Configuration options in *r_rsci_iic_rx_config.h* (1/2)	
RSCI_IIC_CFG_PARAM_CHECKING_ENABLE ※デフォルト値は“1”	パラメータチェック処理をコードに含めるか選択できます。 “0”の場合、パラメータチェック処理をコードから省略します。 “1”の場合、パラメータチェック処理をコードに含めます。
RSCI_IIC_CFG_CHi_INCLUDED i=0、8～11 ※i = 0、8～11 のデフォルト値は“0”	該当チャネルを使用するかを選択できます。 “0”の場合、該当チャネルに関する処理をコードから省略します。 “1”の場合、該当チャネルに関する処理をコードに含めます。 使用時に、使用するチャネルの定義値を”1”に変更してください。
RSCI_IIC_CFG_CHi_BITRATE_BPS i=0、8～11 ※デフォルト値は全て“384000”	ビットレートを設定してください。 384000(384kbit/s)以下を設定してください。 ビットレートは本設定値と RX ファミリ ボードサポートパッケージモジュール(BSP FIT モジュール) のクロックの設定の定義値を元に設定されます。使用する対象デバイスと BSP FIT モジュールのクロックの設定によっては、実際のビットレートが期待するビットレートと異なる場合があります。
RSCI_IIC_CFG_CHi_INT_PRIORITY i=0、8～11 ※デフォルト値は全て“2”	コンディション割り込み、受信割り込み、送信空割り込み、送信完了割り込みの優先レベルを設定してください。 “1”～“15”の範囲で設定してください。
RSCI_IIC_CFG_CHi_DIGITAL_FILTER i=0、8～11 ※デフォルト値は全て“1”	SSCL、SSDA 入力信号のノイズ除去機能を使用するか選択できます。 “0”の場合、ノイズ除去機能を無効にします。 “1”の場合、ノイズ除去機能を有効にします。
RSCI_IIC_CFG_CHi_FILTER_CLOCK i=0、8～11 ※デフォルト値は全て“1”	デジタルノイズフィルタのサンプリングクロックを選択します。 “1”の場合、1分周のクロックをノイズフィルタに使用します。 “2”の場合、2 分周のクロックをノイズフィルタに使用します。 “3”の場合、4 分周のクロックをノイズフィルタに使用します。 “4”の場合、8 分周のクロックをノイズフィルタに使用します。
RSCI_IIC_CFG_CHi_SSDA_DELAY_SELECT i=0、8～11 ※デフォルト値は全て“18”	SSCL端子出力の立ち下がりに対するSSDA端子出力の遅延を選択します。“1”～“31”の範囲で設定してください。 デフォルト値は、内蔵ボーレートジェネレータのクロックソースであるPCLKの周波数が60MHzの場合を基準とした値になっています。 SSDAの遅延時間は内蔵ボーレートジェネレータのクロックソースに応じて遅延時間が増減します。 ビットレートを低速に設定した場合、もしくはPCLKの周波数を低速に設定した場合は、スタートコンディションにおいてSSDAの立ち下がりタイミングがSSCLの立ち下がりタイミングより後に発生することがあります。 システムに応じて、本設定値を見直してください。

Configuration options in *r_rsci_iic_rx_config.h* (2/2)	
RSCI_IIC_CFG_BUS_CHECK_COUNTER ※デフォルト値は“1000”	簡易I²C のAPI 関数のバスチェック処理時の、タイムアウトカウンタ(バス確認回数)を設定できます。 “0xFFFFFFFF”以下の値を設定してください。 バスチェック処理は、 ・簡易I²C 制御機能(R_RSCI_IIC_Control 関数)を使用した各コンディション生成処理実行後に行います。 バスチェック処理では、各コンディション生成処理実行後、タイムアウトカウンタをデクリメントします。“0”になるとタイムアウトと判断し、戻り値でエラー（Busy）を返します。 ※バスロックなどでロックされないようにするためのカウンタであるため、相手デバイスがSCL 端子を“L”ホールドする時間以上になるよう値を設定してください。 「タイムアウト時間(ns) ≒ (1 / ICLK(Hz)) * カウンタ値 * 10」
RSCI_IIC_CFG_PORT_SETTING_PROCESSING ※デフォルト値は“1”	R_RSCI_IIC_CFG_RSCIi_SSCLi_PORT、 R_RSCI_IIC_CFG_RSCIi_SSCLi_BIT、 R_RSCI_IIC_CFG_RSCIi_SSDAi_PORT、 R_RSCI_IIC_CFG_RSCIi_SSDAi_BIT で選択したポートを SSCL、SSDA 端子として使用するための設定処理をコードに含めるか選択できます。 “0”の場合、ポートの設定処理をコードから省略します。 “1”の場合、ポートの設定処理をコードに含めます。 この設定を“0”とした場合でも、上記の4つの定義は設定してください。

Configuration options in *r_rsci_iic_rx_pin_config.h*	
R_RSCI_IIC_CFG_RSCIi_SSCLi_PORT i=0、8～11 ※i = 0 のデフォルト値は'2' ※i = 8 のデフォルト値は'C' ※i = 9 のデフォルト値は'B' ※i = 10 のデフォルト値は'8' ※i = 11 のデフォルト値は'7'	SSCL端子として使用するポートグループを選択します。 '0'～'K' (ASCIIコード)の範囲で設定してください。
R_RSCI_IIC_CFG_RSCIi_SSCLi_BIT i=0、8～11 ※i = 0 のデフォルト値は'1' ※i = 8 のデフォルト値は'6' ※i = 9 のデフォルト値は'6' ※i = 10 のデフォルト値は'1' ※i = 11 のデフォルト値は'6'	SSCL端子として使用する端子を選択します。 '0'～'7' (ASCIIコード)の範囲で設定してください。
R_RSCI_IIC_CFG_RSCIi_SSDAi_PORT i=0、8～11 ※i = 0 のデフォルト値は'2' ※i = 8 のデフォルト値は'C' ※i = 9 のデフォルト値は'B' ※i = 10 のデフォルト値は'8' ※i = 11 のデフォルト値は'7'	SSDA端子として使用するポートグループを選択します。 '0'～'K' (ASCIIコード)の範囲で設定してください。
R_RSCI_IIC_CFG_RSCIi_SSDAi_BIT i=0、8～11 ※i = 0 のデフォルト値は'0' ※i = 8 のデフォルト値は'7' ※i = 9 のデフォルト値は'7' ※i = 10 のデフォルト値は'2' ※i = 11 のデフォルト値は'7'	SSDA端子として使用する端子を選択します。 '0'～'7' (ASCIIコード)の範囲で設定してください。

2.8 コードサイズ

本モジュールのコードサイズを下表に示します。RX200 シリーズ、RX600 シリーズから代表して 1 デバイスずつ掲載しています。

ROM (コードおよび定数) と RAM (グローバルデータ) のサイズは、ビルド時の「2.7 コンパイル時の設定」のコンフィギュレーションオプションによって決まります。掲載した値は、「2.3 サポートされているツールチェーン」の C コンパイラでコンパイルオプションがデフォルト時の参考値です。コンパイルオプションのデフォルトは最適化レベル：2、最適化のタイプ：サイズ優先、データ・エンディアン：リトルエンディアンです。コードサイズは C コンパイラのバージョンやコンパイルオプションにより異なります。

下表の値は下記条件で確認しています。

モジュールリビジョン: r_rsci_iic_rx rev1.00

コンパイラバージョン: Renesas Electronics C/C++ Compiler Package for RX Family V3.07.00

(統合開発環境のデフォルト設定に”-lang = c99”オプションを追加)

GCC for Renesas RX 14.02.00.202505

(統合開発環境のデフォルト設定に” -std=gnu99”オプションを追加)

IAR C/C++ Compiler for Renesas RX version 5.20.01

(統合開発環境のデフォルト設定)

コンフィグレーションオプション: デフォルト設定

ROM、RAM およびスタックのコードサイズ

デバイス	分類		使用メモリ					
			Renesas Compiler		GCC		IAR Compiler	
			パラメータチェックあり	パラメータチェックなし	パラメータチェックあり	パラメータチェックなし	パラメータチェックあり	パラメータチェックなし
RX261	ROM	1 チャネル使用	**4721 バイト**	**4721 バイト**	**5544 バイト**	**5544 バイト**	**6663 バイト**	**6663 バイト**
		2 チャネル使用	**4890 バイト**	**4890 バイト**	**5716 バイト**	**5716 バイト**	**6796 バイト**	**6796 バイト**
	RAM	1 チャネル使用	**41 バイト**		**128 バイト**		**28 バイト**	
		2 チャネル使用	**69 バイト**		**128 バイト**		**44 バイト**	
	スタック（注 1）		**48 バイト**		**-**		**284 バイト**	
RX660	ROM	1 チャネル使用	**4771 バイト**	**4771 バイト**	**5592 バイト**	**5592 バイト**	**6716 バイト**	**6712 バイト**
		2 チャネル使用	**4939 バイト**	**4939 バイト**	**5756 バイト**	**5756 バイト**	**6844 バイト**	**6848 バイト**
	RAM	1 チャネル使用	**41 バイト**		**0 バイト**		**32 バイト**	
		2 チャネル使用	**69 バイト**		**128 バイト**		**48 バイト**	
	スタック（注 1）		**48 バイト**		**-**		**292 バイト**	

注 1.　割り込み関数の最大使用スタックサイズを含みます

2.9 引数

API 関数の引数である構造体を示します。この構造体は API 関数のプロトタイプ宣言とともに r_rsci_iic_rx_if.h で記載されています。

構造体の内容は、通信中に参照、更新されます。このため、通信中(RSCI_IIC_COMMUNICATION)に構造体の内容を書き換えないでください。

```
typedef struct
{
    uint8_t                 rsv2;                /* 予約領域 */
    uint8_t                 rsv1;                /* 予約領域 */
    rsci_iic_ch_dev_status_t dev_sts;            /* デバイス状態フラグ */
    uint8_t                 ch_no;               /* 使用するデバイスのチャネル番号 */
    rsci_iic_callback       callbackfunc;        /* コールバック関数 */
    uint32_t                cnt2nd;              /* 2nd データカウンタ(バイト数) */
    uint32_t                cnt1st;              /* 1st データカウンタ(バイト数) */
    uint8_t *               p_data2nd;           /* 2nd データ格納バッファポインタ */
    uint8_t *               p_data1st;           /* 1st データ格納バッファポインタ */
    uint8_t *               p_slv_adr;           /* スレーブアドレスのバッファポインタ */
} rsci_iic_info_t;
```

2.10 戻り値

API 関数の戻り値を示します。この列挙型は API 関数のプロトタイプ宣言とともに r_rsci_iic_rx_if.h で記載されています。

```
typedef enum                                /* 簡易 I²C バス API のステータスコード
{
    RSCI_IIC_SUCCESS,                       /* 問題なく処理が終了した場合 */
    RSCI_IIC_ERR_LOCK_FUNC,                 /*同じチャネルに対して多重コールされた場合 */
    RSCI_IIC_ERR_INVALID_CHAN,              /* 存在しないチャネルの場合 */
    RSCI_IIC_ERR_INVALID_ARG,               /* 不正な引数の場合 */
    RSCI_IIC_ERR_NO_INIT,                   /* 未初期化状態 */
    RSCI_IIC_ERR_BUS_BUSY,                  /* バスビジー　以下の場合に発生します。*/
                                            /* 通信中に初期化関数または各開始関数をコールした場合 */
                                            /* 同一チャネル上の他のデバイスが通信中に、*/
                                            /* 各開始関数またはアドバンス関数をコールした場合 */
    RSCI_IIC_ERR_OTHER                      /* その他エラー */
} rsci_iic_return_t;
```

2.11 モジュールの追加方法

本モジュールは、使用するプロジェクトごとに追加する必要があります。ルネサスでは、Smart Configurator を使用した(1)、(2)、(4)の追加方法を推奨しています。ただし、Smart Configurator は、一部の RX デバイスのみサポートしています。サポートされていない RX デバイスについては(3)の方法を使用してください。

(1) e² studio 上で Smart Configurator を使用して FIT モジュールを追加する場合
e² studio の Smart Configurator を使用して、自動的にユーザプロジェクトに FIT モジュールを追加します。詳細は、アプリケーションノート「RX スマート・コンフィグレータ ユーザーガイド: e² studio 編 (R20AN0451)」を参照してください。

(2) CS+上で Smart Configurator を使用して FIT モジュールを追加する場合
CS+上で、スタンドアロン版 Smart Configurator を使用して、自動的にユーザプロジェクトに FIT モジュールを追加します。詳細は、アプリケーションノート「RX スマート・コンフィグレータ ユーザーガイド: CS+編 (R20AN0470)」を参照してください。

(3) CS+上で FIT モジュールを追加する場合
CS+上で、手動でユーザプロジェクトに FIT モジュールを追加します。詳細は、アプリケーションノート「RX ファミリ CS+に組み込む方法 Firmware Integration Technology (R01AN1826)」を参照してください。

(4) IAREW 上で Smart Configurator を使用して FIT モジュールを追加する場合
スタンドアロン版 Smart Configurator を使用して、自動的にユーザプロジェクトに FIT モジュールを追加します。詳細は、アプリケーションノート「RX スマート・コンフィグレータ ユーザーガイド: IAREW 編 (R20AN0535)」を参照してください。

2.12 for 文、while 文、do while 文について

本モジュールでは、レジスタの反映待ち処理等で for 文、while 文、do while 文（ループ処理）を使用しています。これらループ処理には、「WAIT_LOOP」をキーワードとしたコメントを記述しています。そのため、ループ処理にユーザがフェイルセーフの処理を組み込む場合は、「WAIT_LOOP」で該当の処理を検索できます。

以下に記述例を示します。

while 文の例：

```
/* WAIT_LOOP */
while(0 == SYSTEM.OSCOVFSR.BIT.PLOVF)
{
    /* The delay period needed is to make sure that the PLL has stabilized. */
}
```

for 文の例：

```
/* Initialize reference counters to 0. */
/* WAIT LOOP */
for (i = 0; i < BSP REG PROTECT TOTAL ITEMS; i++)
{
    g protect counters[i] = 0;
}
```

do while 文の例：

```
/* Reset completion waiting */
do
{
    reg = phy_read(ether_channel, PHY_REG_CONTROL);
    count++;
} while ((reg & PHY_CONTROL_RESET) && (count < ETHER_CFG_PHY_DELAY_RESET)); /* WAIT_LOOP */
```

3. API 関数

R_RSCI_IIC_Open()

この関数は RSCI(簡易 I²C モード) FIT モジュールを初期化する関数です。この関数は他の API 関数を使用する前に実行される必要があります。

Format

```
rsci_iic_return_t R_RSCI_IIC_Open(
        rsci_iic_info_t *  p_rsci_iic_info   /* 構造体データ */
)
```

Parameters

** p_rsci_iic_info*

I²C 通信情報構造体のポインタ。

この構造体のうち、本関数で使用するメンバのみを以下に示します。この構造体の詳細については 2.9 を参照してください。

構造体の内容は、通信中に参照、更新されます。このため、通信中(RSCI_IIC_COMMUNICATION)に構造体の内容を書き換えないでください。

下記のうち API 実行中に値が更新される引数には、'更新あり'と記載しています。

```
rsci_iic_ch_dev_status_t    dev_sts;        /* デバイス状態フラグ(更新あり) */
uint8_t                     ch_no;          /* チャネル番号 */
```

Return Values

```
RSCI_IIC_SUCCESS            /* 問題なく処理が完了した場合 */
RSCI_IIC_ERR_LOCK_FUNC      /* 他のタスクがAPI をロックしている場合 */
RSCI_IIC_ERR_INVALID_CHAN   /* 存在しないチャネルの場合 */
RSCI_IIC_ERR_INVALID_ARG    /* 不正な引数の場合 */
RSCI_IIC_ERR_OTHER          /* 現在の状態に該当しない不正なイベントが発生した場合 */
```

Properties

r_rsci_iic_rx_if.h にプロトタイプ宣言されています。

Description

簡易 I²C バス通信を開始するための初期設定をします。引数で指定した RSCI のチャネルを設定します。チャネルの状態が“未初期化状態 (RSCI_IIC_NO_INIT)”の場合、次の処理を行います。

- 状態フラグの設定
- ポートの入出力設定
- I²C 出力ポートの割り当て
- RSCI のモジュールストップ状態の解除
- API で使用する変数の初期化
- 簡易 I²C バス通信で使用する RSCI レジスタの初期化
- RSCI 割り込みの禁止

Example

```
volatile rsci_iic_return_t ret;
rsci_iic_info_t            rsci_iic_info;

rsci_iic_info.dev_sts = RSCI_IIC_NO_INIT;
rsci_iic_info.ch_no = 0;

ret = R_RSCI_IIC_Open(&rsci_iic_info);
```

Special Notes:

簡易 I²C バス通信を開始するための初期設定の中でビットレートを設定します。ビットレートは、「2.7 コンパイル時の設定」の設定値と BSP FIT モジュールのクロックの設定の定義値を元に設定されます。

使用する対象デバイスと BSP FIT モジュールのクロックの設定によっては、実際のビットレートが期待するビットレートと異なる場合があります。

R_RSCI_IIC_MasterSend()

マスタ送信を開始します。引数に合わせてデータ送信パターンを変更します。ストップコンディション生成まで一括で実施します。

Format

```
rsci_iic_return_t R_RSCI_IIC_MasterSend(
        rsci_iic_info_t *  p_rsci_iic_info   /* 構造体データ */
)
```

Parameters

** p_rsci_iic_info*

I²C 通信情報構造体のポインタ。引数によって、送信パターン(4 パターンあります)を変更できます。各送信パターンの指定方法および引数の設定可能範囲は、「Special Notes」を参照ください。また、送信パターンの波形のイメージは「1.3.2 マスタ送信の処理」を参照ください。

この構造体のうち、本関数で使用するメンバのみを以下に示します。この構造体の詳細については 2.9 を参照してください。

構造体の内容は、通信中に参照、更新されます。このため、通信中(RSCI_IIC_COMMUNICATION)に構造体の内容を書き換えないでください。

スレーブアドレスを設定する際、1 ビット左シフトせずに格納してください。

下記のうち API 実行中に値が更新される引数には、'更新あり'と記載しています。

```
uint8_t *                  p_slv_adr;      /* スレーブアドレスのバッファポインタ */
uint8_t *                  p_data1st;      /* 1st データ格納バッファポインタ(更新あり)*/
uint8_t *                  p_data2nd;      /* 2nd データ格納バッファポインタ(更新あり)*/
rsci_iic_ch_dev_status_t   dev_sts;        /* デバイス状態フラグ(更新あり)*/
uint32_t                   cnt1st;         /* 1st データカウンタ(バイト数)
                                               (パターン 1 のみ更新あり)*/
uint32_t                   cnt2nd;         /* 2nd データカウンタ(バイト数)
                                               (パターン 1、2 のみ更新あり) */
rsci_iic_callback          callbackfunc;   /* コールバック関数 */
uint8_t                    ch_no;          /* チャネル番号 */
```

Return Values

```
RSCI_IIC_SUCCESS            /* 問題なく処理が完了した場合 */
RSCI_IIC_ERR_INVALID_CHAN   /* 存在しないチャネルの場合 */
RSCI_IIC_ERR_INVALID_ARG    /* 不正な引数の場合 */
RSCI_IIC_ERR_NO_INIT        /* 初期設定ができていない場合 (未初期化状態) */
RSCI_IIC_ERR_BUS_BUSY       /* バスビジーの場合 */
RSCI_IIC_ERR_OTHER          /* 現在の状態に該当しない不正なイベントが発生した場合 */
```

Properties

r_rsci_iic_rx_if.h にプロトタイプ宣言されています。

Description

簡易 I²C バスのマスタ送信を開始します。引数で指定した RSCI のチャネル、送信パターンで送信します。チャネルの状態が“アイドル状態”(RSCI_IIC_IDLE)の場合、次の処理を行います。

- 状態フラグの設定
- API で使用する変数の初期化
- RSCI 割り込みの許可
- I²C のリセット解除
- I²C 出力ポートの割り当て
- スタートコンディションの生成

スタートコンディションの生成処理までが正常に終了した時、本関数は戻り値として RSCI_IIC_SUCCESS を返します。

スタートコンディションの生成時に下記条件に該当した時、本関数は戻り値として RSCI_IIC_ERR_BUS_BUSY を返します。(注 1)

- SCL、SDA ラインのいずれかが Low の状態である

送信の処理は、本関数が RSCI_IIC_SUCCESS を返した後発生する割り込み処理の中で順次行われます。
使用する割り込みは、「2.4 使用する割り込みベクタ」を参照ください。
マスタ送信の割り込みの発生タイミングは、「5.2.1 マスタ送信」を参照ください。

送信終了でストップコンディションを発行した後、引数で指定したコールバック関数が呼び出されます。

送信が正常に完了したかどうかは、引数で指定したデバイス状態フラグ、またはチャネル状態フラグ g_rsci_iic_ChStatus[ ]が”RSCI_IIC_FINISH”になっているかどうかで確認することができます。

注1. SCL と SDA 端子が外部回路でプルアップされていない場合、SCL、SDA ラインのいずれかを Low の状態として検出し、RSCI_IIC_ERR_BUS_BUSY を返すことがあります。

Example

- Case1: 送信パターン 1

```
#include <stddef.h>     // NULL definition
#include "platform.h"
#include "r_rsci_iic_rx_if.h"

void main(void);
void Callback_ch0(void);

void main(void)
{
    volatile rsci_iic_return_t ret;
    rsci_iic_info_t            rsci_iic_info;

    uint8_t slave_addr_eeprom[1] = {0x50};  /* Slave address for EEPROM */
    uint8_t access_addr1[1]      = {0x00};  /* 1st data field */
    uint8_t send_data[5]         = {0x81,0x82,0x83,0x84,0x85};

    /* Sets IIC Information (Send pattern 1) */
    rsci_iic_info.p_slv_adr    = slave_addr_eeprom;
    rsci_iic_info.p_data1st    = access_addr1;
    rsci_iic_info.p_data2nd    = send_data;
    rsci_iic_info.dev_sts      = RSCI_IIC_NO_INIT;
    rsci_iic_info.cnt1st       = 1;
    rsci_iic_info.cnt2nd       = 3;
    rsci_iic_info.callbackfunc = &Callback_ch0;
    rsci_iic_info.ch_no        = 0;
```

```
    /* RSCI open */
    ret = R_RSCI_IIC_Open(&rsci_iic_info);
    /* Start Master Send */
    ret = R_RSCI_IIC_MasterSend(&rsci_iic_info);

    if (RSCI_IIC_SUCCESS == ret)
    {
        while(RSCI_IIC_FINISH != rsci_iic_info.dev_sts);
    }
    else
    {
        /* error */
    }

    /* Master send complete */
    while(1);
}

void Callback_ch0(void)
{
    volatile rsci_iic_return_t ret;
    rsci_iic_mcu_status_t      iic_status;
    rsci_iic_info_t            iic_info_ch;

    iic_info_ch.ch_no = 0;
    ret = R_RSCI_IIC_GetStatus(&iic_info_ch, &iic_status);

    if (RSCI_IIC_SUCCESS != ret)
    {
        /* R_RSCI_IIC_GetStatus()関数コールエラー処理 */
    }
    else
    {
        if (1 == iic_status.BIT.NACK)
        {
        /* iic_status のステータスフラグを確認して
           NACK が検出されていた場合の処理 */
        }
    }
}
```

- Case2: 2 つのスレーブデバイス(スレーブ 1、スレーブ 2)への連続送信

```
#include <stddef.h>     // NULL definition
#include <stdio.h>          /* For printf */
#include "platform.h"
#include "r_rsci_iic_rx_if.h"

void main(void);
void Callback_ch0(void);

void main(void)
{
    volatile rsci_iic_return_t ret;
    volatile rsci_iic_info_t            rsci_iic_info_slave1;
    volatile rsci_iic_info_t            rsci_iic_info_slave2;

    uint8_t slave_addr_eeprom[1]  = {0x50}; /* Slave address for EEPROM */
    uint8_t slave_addr_m16c[1]    = {0x01}; /* Slave address for M16C */
    uint8_t write_addr_slave1[1]  = {0x01};     /* 1st data field */
    uint8_t write_addr_slave2[1]  = {0x02};     /* 1st data field */
    uint8_t data_area_slave1[5]   = {0x81,0x82,0x83,0x84,0x85};
    uint8_t data_area_slave2[5]   = {0x18,0x28,0x38,0x48,0x58};

    /* Sets 'Slave 1' Information (Send pattern 1) */
    rsci_iic_info_slave1.p_slv_adr    = slave_addr_eeprom;
    rsci_iic_info_slave1.p_data1st    = write_addr_slave1;
    rsci_iic_info_slave1.p_data2nd    = data_area_slave1;
    rsci_iic_info_slave1.dev_sts      = RSCI_IIC_NO_INIT;
    rsci_iic_info_slave1.cnt1st       = 1;
    rsci_iic_info_slave1.cnt2nd       = 3;
    rsci_iic_info_slave1.callbackfunc = &Callback_ch0;
    rsci_iic_info_slave1.ch_no        = 0;
    uint8_t timeout_cnt1          = 0;

    /* RSCI open */
    ret = R_RSCI_IIC_Open(&rsci_iic_info_slave1);
    /* Start Master Send */
    ret = R_RSCI_IIC_MasterSend(&rsci_iic_info_slave1);

    while((RSCI_IIC_FINISH != rsci_iic_info_slave1.dev_sts) &&
          (RSCI_IIC_NACK != rsci_iic_info_slave1.dev_sts))
    {
        timeout_cnt1 = timeout_cnt1 + 1; /*timeout counter is added*/
        if (timeout_cnt1 >= 100)
        {
            printf("Timeout Happend\n");
            break;
        }
        else
        {
            R_BSP_SoftwareDelay(10, BSP_DELAY_MICROSECS); /*BSP delay*/
        }
    }

    /* Sets 'Slave 2' Information (Send pattern 1) */
    rsci_iic_info_slave2.p_slv_adr    = slave_addr_m16c;
    rsci_iic_info_slave2.p_data1st    = write_addr_slave2;
    rsci_iic_info_slave2.p_data2nd    = data_area_slave2;
    rsci_iic_info_slave2.dev_sts      = RSCI_IIC_NO_INIT;
    rsci_iic_info_slave2.cnt1st       = 1;
```

アクセスするスレーブデバイスごとに情報構造体を書き換えることで複数のスレーブデバイスにアクセスできます。

```
    rsci_iic_info_slave2.cnt2nd       = 3;
    rsci_iic_info_slave2.callbackfunc = &Callback_ch0;
    rsci_iic_info_slave2.ch_no        = 0;
    uint8_t timeout_cnt2          = 0;

    /* Start Master Send */
    ret = R_RSCI_IIC_MasterSend(&rsci_iic_info_slave2);

    while((RSCI_IIC_FINISH != rsci_iic_info_slave2.dev_sts) &&
          (RSCI_IIC_NACK != rsci_iic_info_slave2.dev_sts))
   {
        timeout_cnt2 = timeout_cnt2 + 1; /*timeout counter is added*/
        if (timeout_cnt2 >= 100)
        {
            printf("Timeout Happend\n");
            break;
        }
        else
        {
            R_BSP_SoftwareDelay(10, BSP_DELAY_MICROSECS); /*BSP delay*/
        }
    }
    while(1);
}

void Callback_ch0(void)
{
    volatile rsci_iic_return_t ret;
    rsci_iic_mcu_status_t      iic_status;
    rsci_iic_info_t            iic_info_ch;

    iic_info_ch.ch_no = 0;
    ret = R_RSCI_IIC_GetStatus(&iic_info_ch, &iic_status);

    if (RSCI_IIC_SUCCESS != ret)
    {
        /* R_RSCI_IIC_GetStatus()関数コールエラー処理 */
    }
    else
    {
        if (1 == iic_status.BIT.NACK)
        {
        /* iic_status のステータスフラグを確認して
           NACK が検出されていた場合の処理 */
        }
    }
}
```

Special Notes:
送信パターンごとの引数の設定可能範囲は、下表を参照してください。

構造体メンバ	ユーザ設定可能範囲			
	マスタ送信 パターン 1	マスタ送信 パターン 2	マスタ送信 パターン 3	マスタ送信 パターン 4
*p_slv_adr	スレーブアドレスバッファポインタ	スレーブアドレスバッファポインタ	スレーブアドレスバッファポインタ	FIT_NO_PTR (注 1)
*p_data1st	1st データ格納元アドレス	FIT_NO_PTR (注 1)	FIT_NO_PTR (注 1)	FIT_NO_PTR (注 1)
*p_data2nd	2nd データ （送信データ） バッファポインタ	2nd データ （送信データ） バッファポインタ	FIT_NO_PTR (注 1)	FIT_NO_PTR (注 1)
dev_sts	デバイス状態 フラグ	デバイス状態 フラグ	デバイス状態 フラグ	デバイス状態 フラグ
cnt1st	0000 0001h～ FFFF FFFFh (注 2)	0	0	0
cnt2nd	0000 0001h～ FFFF FFFFh (注 2)	0000 0001h～ FFFF FFFFh (注 2)	0	0
callbackfunc	使用する関数名を指定してください。	使用する関数名を指定してください。	使用する関数名を指定してください。	使用する関数名を指定してください。
ch_no	00h～FFh	00h～FFh	00h～FFh	00h～FFh
rsv1,rsv2	リザーブ（設定無効）	リザーブ（設定無効）	リザーブ（設定無効）	リザーブ（設定無効）

注 1：パターン 2、パターン 3、パターン 4 を使用する場合は、上表のとおり該当の構造体メンバに”FIT_NO_PTR”を入れてください。
注 2：“0”は設定しないでください。

R_RSCI_IIC_MasterReceive()

マスタ受信を開始します。引数に合わせてデータ受信パターンを変更します。ストップコンディション生成まで一括で実施します。

Format

```
rsci_iic_return_t R_RSCI_IIC_MasterReceive(
        rsci_iic_info_t *  p_rsci_iic_info   /* 構造体データ */
)
```

Parameters

** p_rsci_iic_info*

I²C 通信情報構造体のポインタ。引数の設定によって、マスタ受信かマスタ送受信を選択できます。マスタ受信およびマスタ送受信の指定方法と引数の設定可能範囲は「Special Notes」を参照ください。また、受信パターンの波形イメージは「1.3.3 マスタ受信の処理」を参照ください。

この構造体のうち、本関数で使用するメンバのみを以下に示します。この構造体の詳細については 2.9 を参照してください。

構造体の内容は、通信中に参照、更新されます。このため、通信中(RSCI_IIC_COMMUNICATION)に構造体の内容を書き換えないでください。

スレーブアドレスを設定する際、1 ビット左シフトせずに格納してください。

下記のうち API 実行中に値が更新される引数には、'更新あり'と記載しています。

```
uint8_t *                  p_slv_adr;      /* スレーブアドレスのバッファポインタ */
uint8_t *                  p_data1st;      /* 1st データ格納バッファポインタ(更新あり) */
uint8_t *                  p_data2nd;      /* 2nd データ格納バッファポインタ(更新あり) */
rsci_iic_ch_dev_status_t   dev_sts;        /* デバイス状態フラグ(更新あり) */
uint32_t                   cnt1st;         /* 1st データカウンタ(バイト数)
                                               (マスタ送受信のみ更新あり)*/
uint32_t                   cnt2nd;         /* 2nd データカウンタ(バイト数) (更新あり) */
rsci_iic_callback          callbackfunc;   /* コールバック関数 */
uint8_t                    ch_no;          /* チャネル番号 */
```

Return Values

```
RSCI_IIC_SUCCESS            /* 問題なく処理が完了した場合 */
RSCI_IIC_ERR_INVALID_CHAN   /* 存在しないチャネルの場合 */
RSCI_IIC_ERR_INVALID_ARG    /* 不正な引数の場合 */
RSCI_IIC_ERR_NO_INIT        /* 初期設定ができていない場合 (未初期化状態) */
RSCI_IIC_ERR_BUS_BUSY       /* バスビジーの場合 */
RSCI_IIC_ERR_OTHER          /* 現在の状態に該当しない不正なイベントが発生した場合 */
```

Properties

r_rsci_iic_rx_if.h にプロトタイプ宣言されています。

Description

簡易 I²C バスのマスタ受信を開始します。引数で指定した RSCI のチャネル、受信パターンで受信します。チャネルの状態が“アイドル状態”(RSCI_IIC_IDLE) の場合、次の処理を行います。

- 状態フラグの設定
- API で使用する変数の初期化
- RSCI 割り込みの許可
- I²C のリセット解除
- I²C 出力ポートの割り当て
- スタートコンディションの生成

スタートコンディションの生成処理までが正常に終了した時、本関数は戻り値として RSCI_IIC_SUCCESS を返します。
スタートコンディションの生成時に下記条件に該当した時、本関数は戻り値として RSCI_IIC_ERR_BUS_BUSY を返します。(注 1)

- SCL、SDA ラインのいずれかが Low の状態である

受信の処理は、本関数が RSCI_IIC_SUCCESS を返した後発生する割り込み処理の中で順次行われます。
使用する割り込みは、「2.4 使用する割り込みベクタ」を参照ください。
マスタ受信の割り込みの発生タイミングは、「5.2.2 マスタ受信」を参照ください。

受信終了でストップコンディションを発行した後、引数で指定したコールバック関数が呼び出されます。

受信が正常に完了したかどうかは、引数で指定したデバイス状態フラグ、またはチャネル状態フラグ g_rsci_iic_ChStatus[ ]が"RSCI_IIC_FINISH"になっているかどうかで確認することができます。

注1. SCL と SDA 端子が外部回路でプルアップされていない場合、SCL、SDA ラインのいずれかを Low の状態として検出し、RSCI_IIC_ERR_BUS_BUSY を返すことがあります。

Example

```
#include <stddef.h>     // NULL definition
#include "platform.h"
#include "r_rsci_iic_rx_if.h"

void main(void);
void Callback_ch0(void);

void main(void)
{
    volatile rsci_iic_return_t ret;
    rsci_iic_info_t            rsci_iic_info;

    uint8_t slave_addr_eeprom[1] = {0x50}; /* Slave address for EEPROM */
    uint8_t access_addr1[1]      = {0x00}; /* 1st data field           */
    uint8_t store_area[5]        = {0xFF,0xFF,0xFF,0xFF,0xFF};

    /* Sets IIC Information (Ch0) */
    rsci_iic_info.p_slv_adr    = slave_addr_eeprom;
    rsci_iic_info.p_data1st    = access_addr1;
    rsci_iic_info.p_data2nd    = store_area;
    rsci_iic_info.dev_sts      = RSCI_IIC_NO_INIT;
    rsci_iic_info.cnt1st       = 1;
    rsci_iic_info.cnt2nd       = 3;
    rsci_iic_info.callbackfunc = &Callback_ch0;
    rsci_iic_info.ch_no        = 0;

    /* RSCI open */
    ret = R_RSCI_IIC_Open(&rsci_iic_info);
    /* Start Master Receive */
    ret = R_RSCI_IIC_MasterReceive(&rsci_iic_info);

    if (RSCI_IIC_SUCCESS == ret)
    {
        while(RSCI_IIC_FINISH != rsci_iic_info.dev_sts);
    }
    else
```

```
    {
        /* error */
    }

    /* Master receive complete */
    while(1);
}

void Callback_ch0(void)
{
    volatile rsci_iic_return_t ret;
    rsci_iic_mcu_status_t      iic_status;
    rsci_iic_info_t            iic_info_ch;

    iic_info_ch.ch_no = 0;
    ret = R_RSCI_IIC_GetStatus(&iic_info_ch, &iic_status);

    if (RSCI_IIC_SUCCESS != ret)
    {
        /* R_RSCI_IIC_GetStatus()関数コールエラー処理 */
    }
    else
    {
        if (1 == iic_status.BIT.NACK)
        {
        /* iic_status のステータスフラグを確認して
           NACK が検出されていた場合の処理 */
        }
    }
}
```

Special Notes:

受信パターンごとの引数の設定可能範囲は、下表を参照してください。

構造体メンバ	ユーザ設定可能範囲	
	マスタ受信	マスタ送受信
*p_slv_adr	スレーブアドレスバッファポインタ	スレーブアドレスバッファポインタ
*p_data1st	(設定無効)	1st データバッファポインタ
*p_data2nd	2nd データ(受信データ)格納先アドレス	2nd データ(受信データ)格納先アドレス
dev_sts	デバイス状態フラグ	デバイス状態フラグ
cnt1st (注 1)	0	0000 0001h ～FFFF FFFFh
cnt2nd (注 2)	0000 0001h ～FFFF FFFFh	0000 0001h ～FFFF FFFFh
callbackfunc	使用する関数名を指定してください。	使用する関数名を指定してください。
ch_no	00h～FFh	00h～FFh
rsv1,rsv2	リザーブ（設定無効）	リザーブ（設定無効）

注 1：1st データが'0'か'0 以外'かで受信パターンが決まります。
注 2："0"は設定しないでください。

R_RSCI_IIC_Close()

簡易 I²C の通信を終了し、使用していた RSCI の対象チャネルを解放します。

Format

```
rsci_iic_return_t R_RSCI_IIC_Close(
        rsci_iic_info_t *  p_rsci_iic_info   /* 構造体データ */
)
```

Parameters

** p_rsci_iic_info*

I²C 通信情報構造体のポインタ。

この構造体のうち、本関数で使用するメンバのみを以下に示します。この構造体の詳細については 2.9 を参照してください。

構造体の内容は、通信中に参照、更新されます。このため、通信中(RSCI_IIC_COMMUNICATION)に構造体の内容を書き換えないでください。

下記のうち API 実行中に値が更新される引数には、'更新あり'と記載しています。

```
rsci_iic_ch_dev_status_t    dev_sts;        /* デバイスフラグポインタ (更新あり) */
uint8_t                     ch_no;          /* チャネル番号 */
```

Return Values

RSCI_IIC_SUCCESS　*/* 問題なく処理が完了した場合 */*
RSCI_IIC_ERR_INVALID_CHAN　*/* 存在しないチャネルの場合 */*
RSCI_IIC_ERR_INVALID_ARG　*/* 不正な引数の場合 */*

Properties

r_rsci_iic_rx_if.h にプロトタイプ宣言されています。

Description

簡易 I²C バス通信を終了するための設定をします。引数で指定した RSCI のチャネルを無効にします。本関数では次の処理を行います。

- RSCI のモジュールストップ状態への遷移
- I²C 出力ポートの解放
- RSCI 割り込みの禁止

再度通信を開始するには、R_RSCI_IIC_Open ( ) (初期化関数)をコールする必要があります。通信中に強制的に停止した場合、その通信は保証しません。

Example

```
volatile rsci_iic_return_t ret;
rsci_iic_info_t            rsci_iic_info;

rsci_iic_info.ch_no = 0;

ret = R_RSCI_IIC_Close(&rsci_iic_info);
```

Special Notes:

なし

R_RSCI_IIC_GetStatus()

本モジュールの状態を返します。

Format

```
rsci_iic_return_t R_RSCI_IIC_GetStatus(
    rsci_iic_info_t *          p_rsci_iic_info        /* 構造体データ */
    rsci_iic_mcu_status_t *    p_rsci_iic_status              /* 本モジュールのステータス */
);
```

Parameters

** p_rsci_iic_info*

I²C 通信情報構造体のポインタ。

この構造体のうち、本関数で使用するメンバのみを以下に示します。この構造体の詳細については 2.9 を参照してください。

構造体の内容は、通信中に参照、更新されます。このため、通信中(RSCI_IIC_COMMUNICATION)に構造体の内容を書き換えないでください。

```
uint8_t    ch_no;          /* チャネル番号 */
```

**p_rsci_iic_status*

I²C のステータスフラグを格納するアドレスです。引数が"FIT_NO_PTR"のときステータスは返しません。下記構造体で定義しているメンバで指定します。

```
typedef union
{
    uint32_t    LONG;
    struct      st_rsci_iic_status_flag
    {
        uint32_t  rsv :27;        /* 予約ビット */
        uint32_t  SCLI:1;         /* SSCL ピンレベル */
        uint32_t  SDAI:1;         /* SSDA ピンレベル */
        uint32_t  NACK :1;        /* NACK 検出フラグ */
        uint32_t  TRS :1;         /* 送受信モードレベル */
        uint32_t  BSY :1;         /* バス状態フラグ */
    }BIT;
} rsci_iic_mcu_status_t;
```

Return Values

```
RSCI_IIC_SUCCESS              /* 問題なく処理が完了した場合 */
RSCI_IIC_ERR_INVALID_CHAN     /* 存在しないチャネルの場合 */
RSCI_IIC_ERR_INVALID_ARG      /* 不正な引数の場合 */
RSCI_IIC_ERR_OTHER            /* 現在の状態に該当しない不正なイベントが発生した場合 */
```

Properties

r_rsci_iic_rx_if.h にプロトタイプ宣言されています。

Description

本モジュールの状態を返します。

引数で指定した RSCI のチャネルの状態を、レジスタの読み出し、端子レベルの読み出し、変数の読み出しなどにより取得し、32 ビットの構造体で戻り値として返します。

Example

```
volatile rsci_iic_return_t ret;
rsci_iic_info_t            rsci_iic_info;
rsci_iic_mcu_status_t      iic_status;

rsci_iic_info.ch_no = 0;

ret = R_RSCI_IIC_GetStatus(&rsci_iic_info, &iic_status);
```

Special Notes:

以下にステータスフラグの配置を示します。

b31 - b16
Reserve
Reserve
Rsv
Always 0

b15 – b8
Reserve
Reserve
Rsv
Always 0

b7– b5	b4	b3	b2	b1	b0
Reserve	Pin Level		Event detection	Mode	Bus state
Reserve	SSCL pin level	SSDA pin Level	NACK detection	Send/Receive mode	Bus busy/ready
Rsv	SCLI	SDAI	NACK	TRS	BSY
Always 0	0:Low level 1:High level		0:Not detected 1:Detected	0:Receive 1:Transmit	0:Idle 1:Busy

R_RSCI_IIC_Control()

各コンディション出力、NACK 出力、SSCL クロックのワンショット出力、および本モジュールリセットを行う関数です。主に通信エラー時に使用してください。

Format

```
rsci_iic_return_t R_RSCI_IIC_Control(
        r_rsci_iic_info_t *         p_rsci_iic_info   /* 構造体データ */
        rsci_iic_ctrl_ptn_t         ctrl_ptn          /* 出力パターン */
)
```

Parameters

* *p_rsci_iic_info*

I²C 通信情報構造体のポインタ。

この構造体のうち、本関数で使用するメンバのみを以下に示します。この構造体の詳細については 2.9 を参照してください。

構造体の内容は、通信中に参照、更新されます。このため、通信中(RSCI_IIC_COMMUNICATION)に構造体の内容を書き換えないでください。

下記のうち API 実行中に値が更新される引数には、'更新あり'と記載しています。

```
rsci_iic_ch_dev_status_t    dev_sts;        /* デバイスフラグポインタ (更新あり) */
uint8_t                     ch_no;          /* チャネル番号 */
```

ctrl_ptn

生成波形パターンの設定をします。複数選択する場合は、"|"(OR)を用いてください。

- 複数選択が可能なパターン

・'RSCI_IIC_GEN_START_CON', 'RSCI_IIC_GEN_RESTART_CON', 'RSCI_IIC_GEN_STOP_CON'の 3 つは同時指定可能です。
3 つを組み合わせて選択する場合は、上記順番で実施されます。

・'RSCI_IIC_GEN_SSDA_HI_Z', 'RSCI_IIC_GEN_SSCL_ONESHOT'の 2 つは同時指定可能です。

```
typedef uint8_t rsci_iic_ctrl_ptn_t;

#define RSCI_IIC_GEN_START_CON          (rsci_iic_ctrl_ptn_t)(0x01)
                                                /* スタートコンディションの生成 */
#define RSCI_IIC_GEN_STOP_CON           (rsci_iic_ctrl_ptn_t)(0x02)
                                                /* ストップコンディションの生成 */
#define RSCI_IIC_GEN_RESTART_CON        (rsci_iic_ctrl_ptn_t)(0x04)
                                                /* リスタートコンディションの生成 */
#define RSCI_IIC_GEN_SSDA_HI_Z          (rsci_iic_ctrl_ptn_t)(0x08)
                                                /* SSDA 端子から Hi-z 出力 */
#define RSCI_IIC_GEN_SSCL_ONESHOT       (rsci_iic_ctrl_ptn_t)(0x10)
                                                /* SSCL クロックのワンショット出力 */
#define RSCI_IIC_GEN_RESET              (rsci_iic_ctrl_ptn_t)(0x20)
                                                /* 簡易 I2C バスのモジュールリセット */
```

Return Values

```
RSCI_IIC_SUCCESS            /* 問題なく処理が完了した場合 */
RSCI_IIC_ERR_INVALID_CHAN   /* 存在しないチャネルの場合 */
RSCI_IIC_ERR_INVALID_ARG    /* 不正な引数の場合 */
RSCI_IIC_ERR_BUS_BUSY       /* バスビジーの場合 */
RSCI_IIC_ERR_OTHER          /* 現在の状態に該当しない不正なイベントが発生した場合 */
```

Properties

r_rsci_iic_rx_if.h にプロトタイプ宣言されています。

Description

簡易 I²C バスの制御信号を出力します。引数で指定した各コンディション出力、SSDA 端子から Hi-z 出力、SSCL クロックのワンショット出力、および簡易 I²C バスのモジュールリセットを行います。

Example

```
volatile rsci_iic_return_t ret;
rsci_iic_info_t            rsci_iic_info;

rsci_iic_info.ch_no = 0;

/* Output an extra SSCL clock cycle after changes the SSDA pin in a high-
impedance state */
ret = R_RSCI_IIC_Control(&rsci_iic_info, RSCI_IIC_GEN_SSDA_HI_Z |
RSCI_IIC_SSCL_ONESHOT);
```

Special Notes:

なし

R_RSCI_IIC_GetVersion()

本モジュールのバージョンを返します。

Format
uint32_t R_RSCI_IIC_GetVersion(void)

Parameters
なし

Return Values
バージョン番号

Properties
r_rsci_iic_rx_if.h にプロトタイプ宣言されています。

Description
本関数は、現在インストールされている RSCI(簡易 I²C モード) FIT モジュールのバージョンを返します。バージョン番号はコード化されています。最初の 2 バイトがメジャーバージョン番号、後の 2 バイトがマイナーバージョン番号です。例えば、バージョンが 4.25 の場合、戻り値は‘0x00040019’となります。

Example

```
uint32_t version;

version = R_RSCI_IIC_GetVersion();
```

Special Notes:
なし。

4. 端子設定

RSCI (簡易 I²C モード) FIT モジュールを使用するためには、マルチファンクションピンコントローラ(MPC)で周辺機能の入出力信号を端子に割り付ける（以下、端子設定と称す）必要があります。

RSCI (簡易 I²C モード) FIT モジュールは、コンフィグレーションオプションの RSCI_IIC_CFG_PORT_SETTING_PROCESSING の設定により、R_RSCI_IIC_Open／R_RSCI_IIC_MasterSend／R_RSCI_IIC_MasterReceive／R_RSCI_IIC_Close／R_RSCI_IIC_Control 関数の中で端子設定するかを選択できます。

コンフィグレーションオプションの詳細は、「2.7 コンパイル時の設定」を参照ください。

e² studio の場合は「Smart Configurator」の端子設定機能を使用することができます。Smart Configurator の端子機能を使用すると、端子設定画面で選択した端子を使用することができます。選択した端子情報は r_rsci_iic_pin_config.h に反映され、表 **4.1** に示すマクロ定義の値が選択した端子に応じた値に上書きされます。

表 4.1 端子設定マクロ定義

選択したチャネル	選択した端子	マクロ定義
チャネル 0	SSCL000 端子	R_RSCI_IIC_CFG_RSCI0_SSCL000_PORT R_RSCI_IIC_CFG_RSCI0_SSCL000_BIT
	SSDA000 端子	R_RSCI_IIC_CFG_RSCI0_SSDA000_PORT R_RSCI_IIC_CFG_RSCI0_SSDA000_BIT
チャネル 8	SSCL008 端子	R_RSCI_IIC_CFG_RSCI8_SSCL008_PORT R_RSCI_IIC_CFG_RSCI8_SSCL008_BIT
	SSDA008 端子	R_RSCI_IIC_CFG_RSCI8_SSDA008_PORT R_RSCI_IIC_CFG_RSCI8_SSDA008_BIT
チャネル 9	SSCL009 端子	R_RSCI_IIC_CFG_RSCI9_SSCL009_PORT R_RSCI_IIC_CFG_RSCI9_SSCL009_BIT
	SSDA009 端子	R_RSCI_IIC_CFG_RSCI9_SSDA009_PORT R_RSCI_IIC_CFG_RSCI9_SSDA009_BIT
チャネル 10	SSCL010 端子	R_RSCI_IIC_CFG_RSCI10_SSCL010_PORT R_RSCI_IIC_CFG_RSCI10_SSCL010_BIT
	SSDA010 端子	R_RSCI_IIC_CFG_RSCI10_SSDA010_PORT R_RSCI_IIC_CFG_RSCI10_SSDA010_BIT
チャネル 11	SSCL011 端子	R_RSCI_IIC_CFG_RSCI11_SSCL011_PORT R_RSCI_IIC_CFG_RSCI11_SSCL011_BIT
	SSDA011 端子	R_RSCI_IIC_CFG_RSCI11_SSDA011_PORT R_RSCI_IIC_CFG_RSCI11_SSDA011_BIT

r_rsci_iic_pin_config.h で選択した端子は、R_RSCI_IIC_MasterSend／R_RSCI_IIC_MasterReceive／R_RSCI_IIC_Control 関数呼び出し後、周辺機能端子として SSCL 端子、SSDA 端子となります。

周辺機能端子の割り付けは R_RSCI_IIC_MasterSend／R_RSCI_IIC_MasterReceive／R_RSCI_IIC_Control 関数で実行した通信動作を完了するか、もしくは R_RSCI_IIC_Close 関数が呼び出されると解除され、端子は汎用入出力端子（入力状態）になります。

なお、SSCL 端子、SSDA 端子は外付け抵抗でプルアップ処理を行ってください。

もし RSCI_IIC_CFG_PORT_SETTING_PROCESSING の設定で本モジュール内の端子設定処理を使用しない場合は、R_RSCI_IIC_Open 関数を呼び出した後、その他の API を呼び出す前にユーザ処理で使用する端子の設定を行ってください。

5. 付録

5.1 通信の実現方法

本 API では、スタートコンディションやスレーブアドレス送信などの処理を 1 つのプロトコルとして管理しており、このプロトコルを組み合わせることで通信を実現します。

5.1.1 API 動作の状態

表 5.1 にプロトコル制御を実現するための状態を定義します。

表 5.1 プロトコル制御のための状態一覧 (enum rsci_iic_api_status_t)

No	定数名	内容
STS0	RSCI_IIC_STS_NO_INIT	未初期化状態
STS1	RSCI_IIC_STS_IDLE	アイドル状態
STS2	RSCI_IIC_STS_ST_COND_WAIT	スタートコンディション生成完了待ち状態
STS3	RSCI_IIC_STS_SEND_SLVADR_W_WAIT	スレーブアドレス[Write]送信完了待ち状態
STS4	RSCI_IIC_STS_SEND_SLVADR_R_WAIT	スレーブアドレス[Read]送信完了待ち状態
STS5	RSCI_IIC_STS_SEND_DATA_WAIT	データ送信完了待ち状態
STS6	RSCI_IIC_STS_RECEIVE_DATA_WAIT	データ受信完了待ち状態
STS7	RSCI_IIC_STS_SP_COND_WAIT	ストップコンディション生成完了待ち状態

5.1.2 API 動作中のイベント

表 5.2 にプロトコル制御時に発生するイベントを定義します。割り込みだけでなく、本モジュールが提供するインタフェースがコールされた際も、イベントとして定義します。

表 5.2 プロトコル制御のためのイベント一覧 (enum rsci_iic_api_event_t)

No	イベント	イベントの定義
EV0	RSCI_IIC_EV_INIT	rsci_iic_init_driver()コール
EV1	RSCI_IIC_EV_GEN_START_COND	rsci_iic_generate_start_cond()コール
EV2	RSCI_IIC_EV_INT_START	STI 割り込み発生(割り込みフラグ：START)
EV3	RSCI_IIC_EV_INT_ADD	TXI 割り込み発生
EV4	RSCI_IIC_EV_INT_SEND	TXI 割り込み発生
EV5	RSCI_IIC_EV_INT_STOP	STI 割り込み発生(割り込みフラグ：STOP)
EV6	RSCI_IIC_EV_INT_NACK	STI 割り込み発生(割り込みフラグ：NACK)

5.1.3　プロトコル状態遷移

本モジュールでは、提供インタフェースのコール、または、RSCI(簡易 I²C モード)の割り込み発生をトリガに状態が遷移します。図 5.1～図 5.4 に各プロトコルの状態遷移を示します。

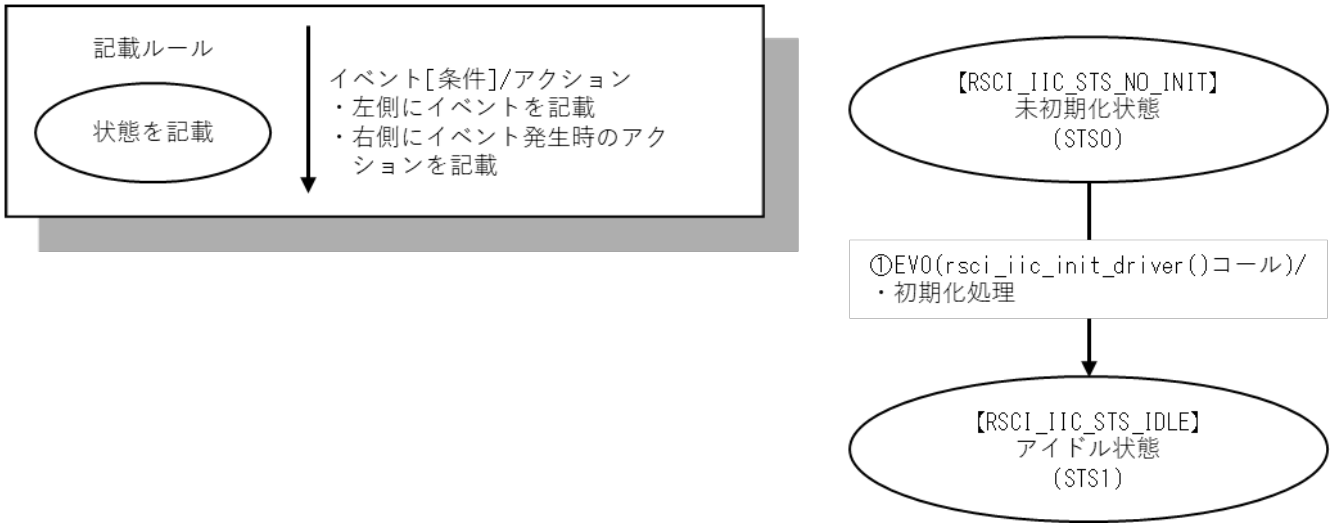


図 5.1 初期化状態遷移図

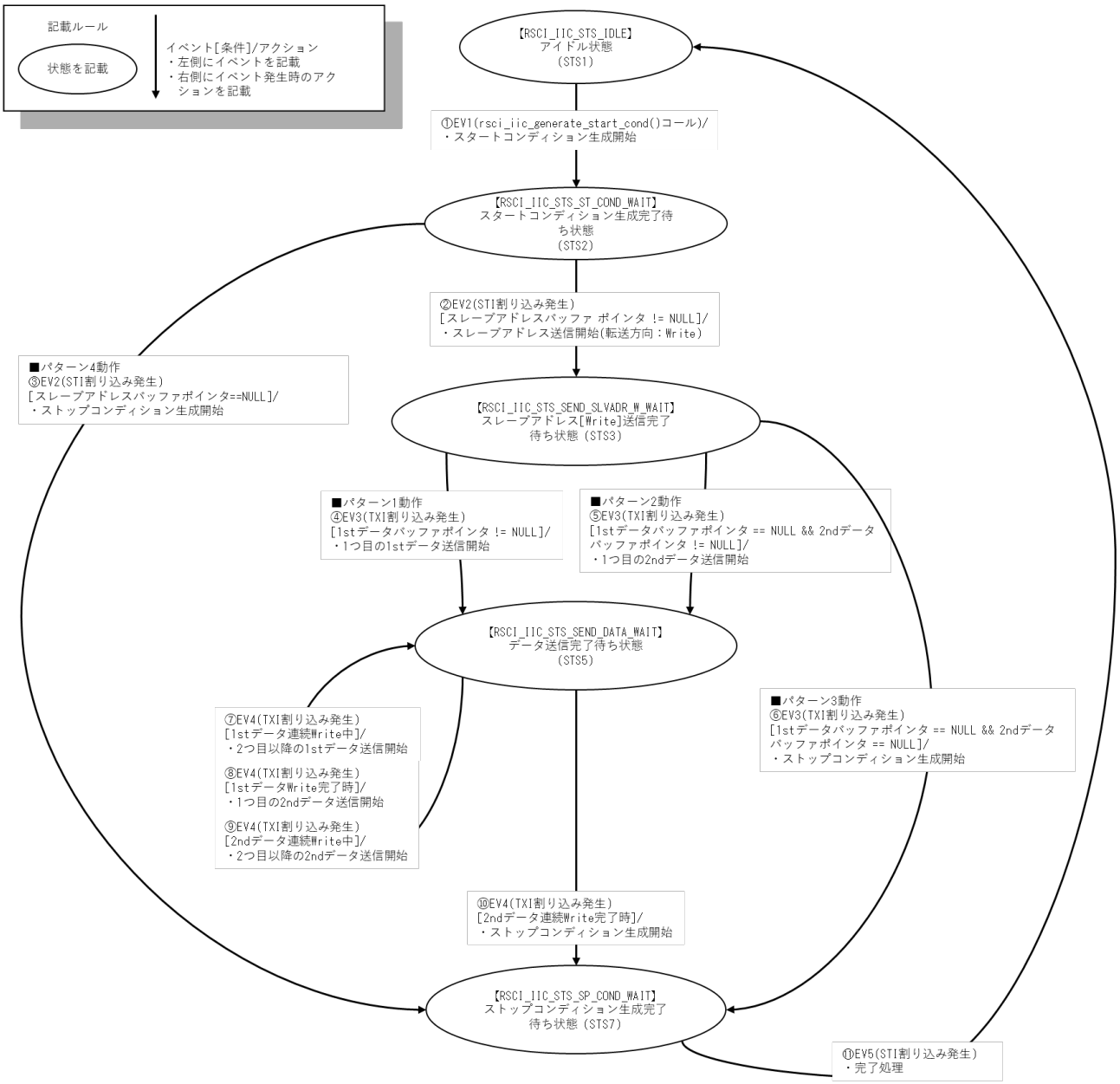


図 5.2 マスタ送信 状態遷移図

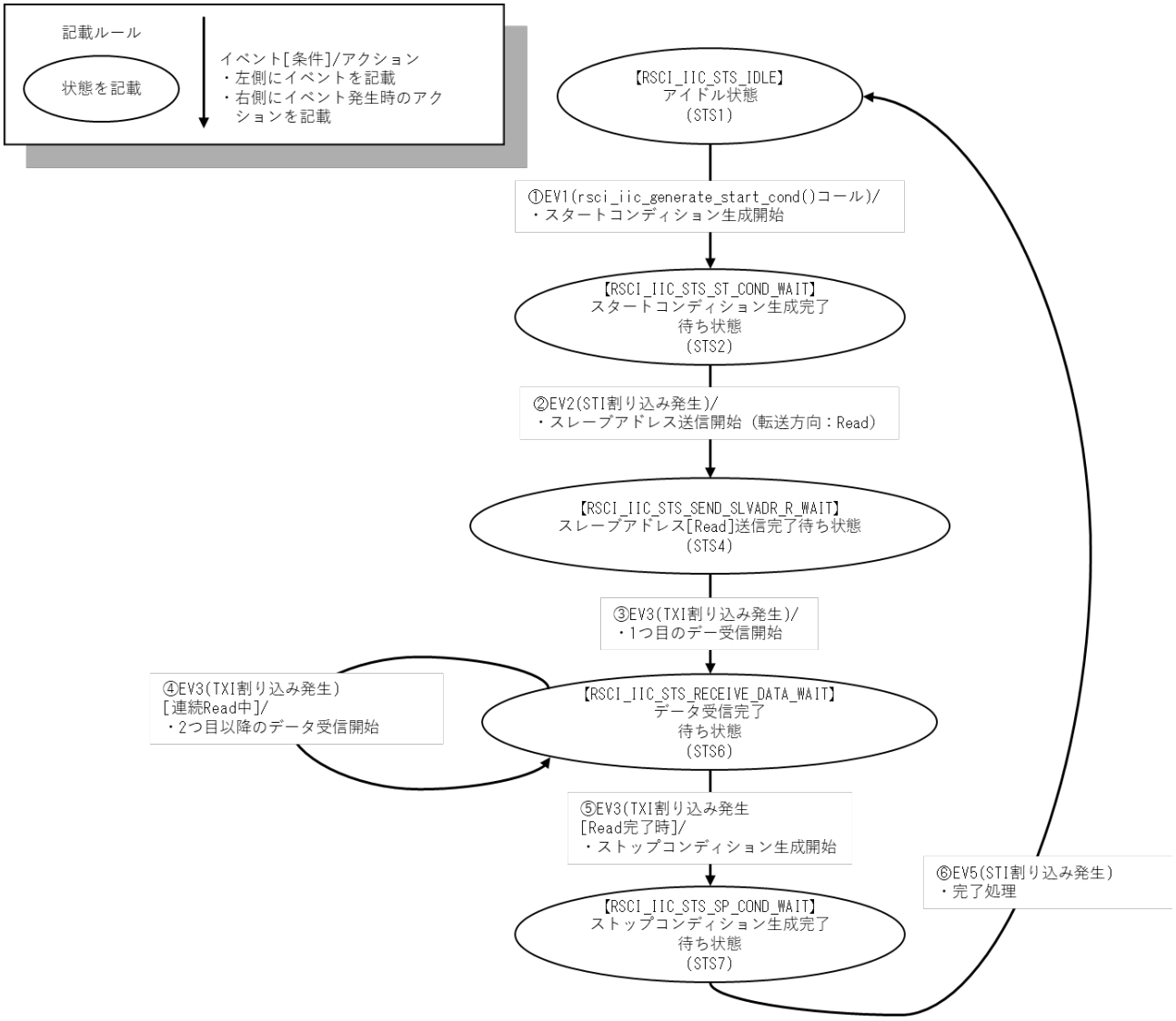


図 5.3 マスタ受信 状態遷移図

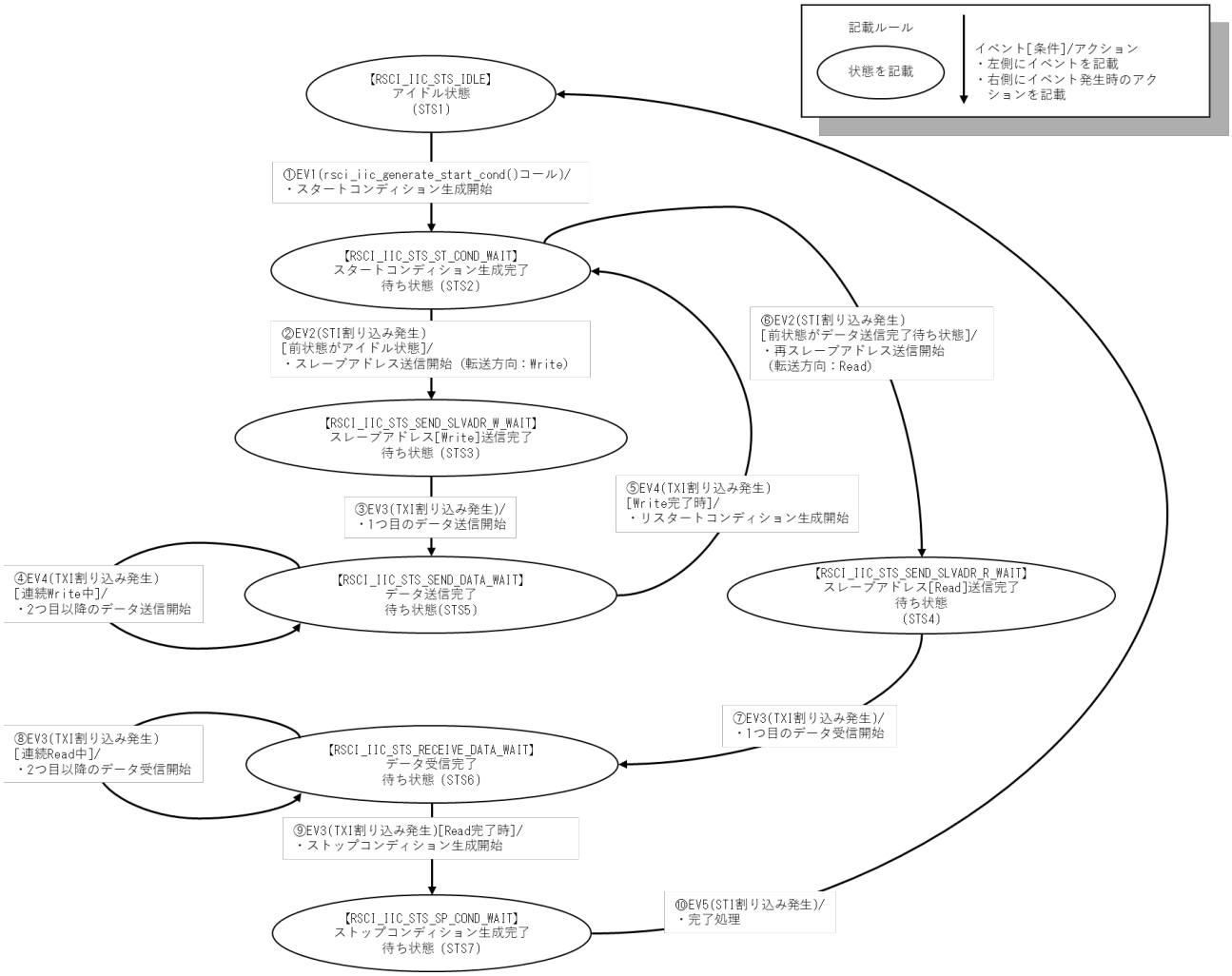


図 5.4 マスタ送受信 状態遷移図

5.1.4　プロトコル状態遷移表

表 5.1 の各状態で、表 5.2 のイベントが発生した際に動作する処理を、表 5.3 の状態遷移表に定義します。Func については表 5.4 を参照してください。

表 5.3 プロトコル状態遷移表(gc_rsci_iic_mtx_tbl[ ] [ ])

状態 ＼ イベント		EV0	EV1	EV2	EV3	EV4	EV5	EV6
STS0	未初期化状態 【RSCI_IIC_STS_NO_INIT】	Func0	ERR	ERR	ERR	ERR	ERR	ERR
STS1	アイドル状態 【RSCI_IIC_STS_IDLE】	ERR	Func1	ERR	ERR	ERR	ERR	ERR
STS2	スタートコンディション生成完了待ち状態 【RSCI_IIC_STS_ST_COND_WAIT】	ERR	ERR	Func2	ERR	ERR	ERR	Func7
STS3	スレーブアドレス[Write]送信完了待ち状態 【RSCI_IIC_STS_SEND_SLVADR_W_WAIT】	ERR	ERR	ERR	Func3	ERR	ERR	Func7
STS4	スレーブアドレス[Read]送信完了待ち状態 【RSCI_IIC_STS_SEND_SLVADR_R_WAIT】	ERR	ERR	ERR	Func3	ERR	ERR	Func7
STS5	データ送信完了待ち状態 【RSCI_IIC_STS_SEND_DATA_WAIT】	ERR	ERR	ERR	ERR	Func4	ERR	Func7
STS6	データ受信完了待ち状態 【RSCI_IIC_STS_RECEIVE_DATA_WAIT】	ERR	ERR	ERR	Func5	ERR	ERR	Func7
STS7	ストップコンディション生成完了待ち状態 【RSCI_IIC_STS_SP_COND_WAIT】	ERR	ERR	ERR	ERR	ERR	Func6	Func7

備考：ERR は RSCI_IIC_ERR_OTHER を表します。ある状態で意図しないイベントが通知された場合には、すべてエラー処理を行います。

5.1.5　プロトコル状態遷移登録関数

表 5.4 に状態遷移表に登録されている関数を定義します。

表 5.4 プロトコル状態遷移登録関数一覧

処理	関数名	概要
Func0	rsci_iic_init_driver()	初期設定処理
Func1	rsci_iic_generate_start_cond()	スタートコンディション生成処理
Func2	rsci_iic_after_gen_start_cond()	スタートコンディション生成後処理
Func3	rsci_iic_after_send_slvadr()	スレーブアドレス送信完了後処理
Func4	rsci_iic_write_data_sending()	データ送信処理
Func5	rsci_iic_read_data_receiving()	データ受信処理
Func6	rsci_iic_release()	通信完了処理
Func7	rsci_iic_nack()	NACK エラー処理

5.1.6　状態遷移時の各フラグの状態

1) 各チャネル状態管理

チャネル状態フラグ g_rsci_iic_ChStatus[]により、1 つのバス上に接続された複数スレーブデバイスの排他制御を行います。

本フラグは、各チャネルに対して 1 つ存在し、グローバル変数で管理します。本モジュールの初期化処理を完了し、対象バスで通信が行われていない場合、本フラグは“RSCI_IIC_IDLE/RSCI_IIC_FINISH/RSCI_IIC_NACK”(アイドル状態(通信可能))となり、通信が可能です。通信中の本フラグの状態は、“RSCI_IIC_COMMUNICATION”(通信中)になります。通信開始時、必ず本フラグの確認を行うため、通信中に同一チャネル上の他デバイスの通信を開始しません。本フラグをチャネルごとに管理することで、複数チャネルの同時通信を実現します。

2) 各デバイス状態管理

I²C 通信情報構造体メンバのデバイス状態フラグ(dev_sts)により、同一チャネル上の複数のスレーブデバイスの制御を行うことができます。デバイス状態フラグには、そのデバイスの通信状態が格納されます。

表 5.5 に状態遷移時の各フラグの状態を示します。

表 5.5 状態遷移時の各フラグの状態一覧

状態	チャネル状態フラグ	デバイス状態フラグ (通信のデバイス)	I²C プロトコルの動作モード	プロトコル制御の現状態
	g_rsci_iic_ChStatus[]	I²C 通信情報構造体 dev_sts	内部通信情報構造体 api_Mode	内部通信情報構造体 api_N_status
未初期化状態	RSCI_IIC_NO_INIT	RSCI_IIC_NO_INIT	RSCI_IIC_MODE_NONE	RSCI_IIC_STS_NO_INIT
アイドル状態	RSCI_IIC_IDLE RSCI_IIC_FINISH RSCI_IIC_NACK	RSCI_IIC_IDLE RSCI_IIC_FINISH RSCI_IIC_NACK	RSCI_IIC_MODE_NONE	RSCI_IIC_STS_IDLE
通信中 (マスタ送信)	RSCI_IIC_COMMUNICATION	RSCI_IIC_COMMUNICATION	RSCI_IIC_MODE_SEND	RSCI_IIC_STS_ST_COND_WAIT
				RSCI_IIC_STS_SEND_SLVADR_W_WAIT
				RSCI_IIC_STS_SEND_DATA_WAIT
				RSCI_IIC_STS_SP_COND_WAIT
通信中 (マスタ受信)	RSCI_IIC_COMMUNICATION	RSCI_IIC_COMMUNICATION	RSCI_IIC_MODE_RECEIVE	RSCI_IIC_STS_ST_COND_WAIT
				RSCI_IIC_STS_SEND_SLVADR_R_WAIT
				RSCI_IIC_STS_RECEIVE_DATA_WAIT
				RSCI_IIC_STS_SP_COND_WAIT
通信中 (マスタ送受信)	RSCI_IIC_COMMUNICATION	RSCI_IIC_COMMUNICATION	RSCI_IIC_MODE_SEND_RECEIVE	RSCI_IIC_STS_ST_COND_WAIT
				RSCI_IIC_STS_SEND_SLVADR_W_WAIT
				RSCI_IIC_STS_SEND_SLVADR_R_WAIT
				RSCI_IIC_STS_SEND_DATA_WAIT
				RSCI_IIC_STS_RECEIVE_DATA_WAIT
				RSCI_IIC_STS_SP_COND_WAIT
エラー	RSCI_IIC_ERROR	RSCI_IIC_ERROR	-	-

5.2　割り込み発生タイミング

以下に本モジュールの割り込みタイミングを示します。

備考　ST　：スタートコンディション
AD6-AD0：スレーブアドレス
/W　：転送方向ビット“0”(Write)
R　：転送方向ビット“1”(Read)
/ACK　：Acknowledge“0”
NACK　：Acknowledge“1”
D7-D0　：データ
RST　：リスタートコンディション
SP　：ストップコンディション

5.2.1　マスタ送信

(1)　パターン 1

ST	AD6-AD0	/W	/ACK	D7-D0	/ACK	D7-D0	/ACK	SP

▲1　▲2　▲3　▲4　▲5

▲1：STI(START)割り込み・・・スタートコンディション検出
▲2：TXI 割り込み・・・アドレス送信完了(転送方向ビット：Write)　※1
▲3：TXI 割り込み・・・データ送信完了(1st データ)　※1
▲4：TXI 割り込み・・・データ送信完了(2nd データ)　※1
▲5：STI(STOP)割り込み・・・ストップコンディション検出

(2)　パターン 2

ST	AD6-AD0	/W	/ACK	D7-D0	/ACK	SP

▲1　▲2　▲3　▲4

▲1：STI(START)割り込み・・・スタートコンディション検出
▲2：TXI 割り込み・・・アドレス送信完了(転送方向ビット：Write)　※1
▲3：TXI 割り込み・・・データ送信完了(2nd データ)　※1
▲4：STI(STOP)割り込み・・・ストップコンディション検出

(3)　パターン 3

ST	AD6-AD0	/W	/ACK	SP

▲1　　▲2　　▲3

▲1：STI(START)割り込み・・・スタートコンディション検出

▲2：TXI 割り込み・・・アドレス送信完了(転送方向ビット：Write)　※1

▲3：STI(STOP)割り込み・・・ストップコンディション検出

(4)　パターン 4

ST	SP

▲1　　▲2

▲1：STI(START)割り込み・・・スタートコンディション検出

▲2：STI(STOP)割り込み・・・ストップコンディション検出

5.2.2　マスタ受信

ST	AD6-AD0	R	/ACK	D7-D0	/ACK	D7-D0	NACK	SP

▲1　　▲2　　▲3　　▲4　　▲5

▲1：STI(START)割り込み・・・スタートコンディション検出

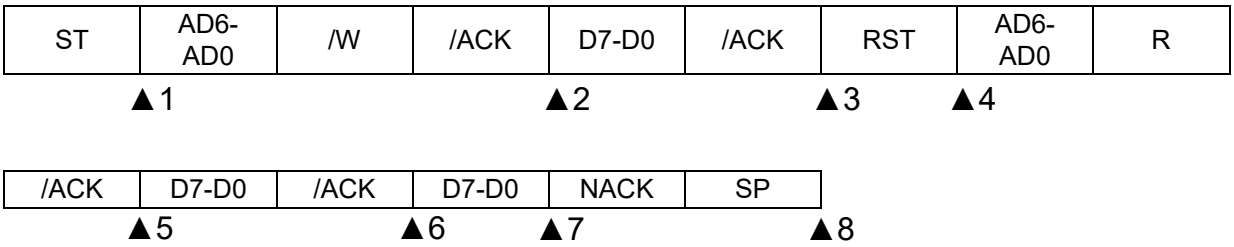
▲2：TXI 割り込み・・・アドレス送信完了(転送方向ビット：Read)　※1

▲3：TXI 割り込み・・・最終データ－1 受信完了(2nd データ)　※1

▲4：TXI 割り込み・・・最終データ受信完了(2nd データ)　※2

▲5：STI(STOP)割り込み・・・ストップコンディション検出

5.2.3　マスタ送受信



▲1：STI(START)割り込み・・・スタートコンディション検出

▲2：TXI 割り込み・・・アドレス送信完了(転送方向ビット：Write)　※1

▲3：TXI 割り込み・・・データ送信完了(1st データ)　※1

▲4：STI(START)割り込み・・・リスタートコンディション検出

▲5：TXI 割り込み・・・アドレス送信完了(転送方向ビット：Read)　※1

▲6：TXI 割り込み・・・最終データ－1 受信完了(2nd データ)　※1

▲7：TXI 割り込み・・・最終データ受信完了(2nd データ)　※2

▲8：STI(STOP)割り込み・・・ストップコンディション検出

※1：9 クロック目の立ち上がりエッジで要求の発生

※2：8 クロック目の立ち上がりで要求の発生

5.3 動作確認環境詳細

本モジュールの動作確認環境を以下に示します。

表 5.6 動作確認環境(Rev.1.00)

項目	内容
統合開発環境	ルネサス エレクトロニクス製 e² studio Version 2025-12 IAR Embedded Workbench for Renesas RX 5.20.01
C コンパイラ	ルネサス エレクトロニクス製 C/C++ compiler for RX family V.3.07.00 コンパイルオプション：統合開発環境のデフォルト設定に以下のオプションを追加 -lang = c99
	GCC for Renesas RX 14.02.00.202505 コンパイルオプション：統合開発環境のデフォルト設定に以下のオプションを追加 -std=gnu99 リンクオプション：「Optimize size（サイズ最適化）(-Os)」を使用する場合、統合開発環境のデフォルト設定に以下のオプションを追加 -Wl,--no-gc-sections これは、FIT 周辺機器モジュール内で宣言されている割り込み関数をリンカが誤って破棄（discard）することを回避（work around）するための対策です。
	IAR C/C++ Compiler for Renesas RX version 5.20.01 コンパイルオプション：統合開発環境のデフォルト設定
エンディアン	ビッグエンディアン/リトルエンディアン
モジュールのリビジョン	Rev.1.00
使用ボード	Evaluation Kit for RX261（型名：RTK5EK2610S00011BJ） Evaluation Kit for RX671（型名：RTK5EK6710S00001BE） Renesas Starter Kit for RX660（型名：RTK556609HC00000BE） Renesas Flexible Motor Control Kit for RX26T（型名：RTK0EMXE70S00020BJ）

5.4 トラブルシューティング

(1) Q：本 FIT モジュールをプロジェクトに追加しましたが、ビルド実行すると「Could not open source file "platform.h"」エラーが発生します。

A：FIT モジュールがプロジェクトに正しく追加されていない可能性があります。プロジェクトへの追加方法をご確認ください。

- CS+を使用している場合
  アプリケーションノート RX ファミリ CS+に組み込む方法 Firmware Integration Technology (R01AN1826)」
- e² studio を使用している場合
  アプリケーションノート RX ファミリ e² studio に組み込む方法 Firmware Integration Technology (R01AN1723)」

また、本 FIT モジュールを使用する場合、ボードサポートパッケージ FIT モジュール(BSP モジュール)もプロジェクトに追加する必要があります。BSP モジュールの追加方法は、アプリケーションノート「ボードサポートパッケージモジュール(R01AN1685)」を参照してください。

(2) Q：本 FIT モジュールをプロジェクトに追加しましたが、ビルド実行すると「This MCU is not supported by the current r_rsci_iic_rx module.」エラーが発生します。

A：追加した FIT モジュールがユーザプロジェクトのターゲットデバイスに対応していない可能性があります。追加した FIT モジュールの対象デバイスを確認してください。

(3) Q：本 FIT モジュールをプロジェクトに追加しましたが、ビルド実行すると「ERROR – RSCI_IIC_CFG_xxx_xxx - …」エラーが発生します。

A：“r_rsci_iic_rx_config.h”ファイルの設定値が間違っている可能性があります。
“r_rsci_iic_rx_config.h”ファイルを確認して正しい値を設定してください。詳細は「2.7 コンパイル時の設定」を参照してください。

6. サンプルコード

6.1 1 つのチャネルで 1 つのスレーブデバイスに連続アクセスする場合の例

RSCI の 1 つのチャネルを簡易 I²C モードで使用し、1 つのスレーブデバイスに対して、書き込む場合のサンプルコードを示します。

次の(1)～(5)の順に動作します。

(1) RSCI の ch0 を簡易 I²C FIT モジュールで使用可能にするため、R_RSCI_IIC_Open 関数を実行する。

(2) デバイス A に 3 バイトのデータを書き込むため、R_RSCI_IIC_MasterSend 関数を実行する。

(3) 送信データの更新

(4) デバイス A に 3 バイトのデータを書き込むため、R_RSCI_IIC_MasterSend 関数を実行する。

(5) RSCI の ch0 を簡易 I²C FIT モジュールから解放するため、R_RSCI_IIC_Close 関数を実行する。

プログラムの説明で、
次の略称を使用しています。
ST:スタートコンディション
SP:ストップコンディション

```
#include <stddef.h> // NULL definition
#include "platform.h"
#include "r_rsci_iic_rx_if.h"

/* NACK 検出時のリトライ回数の定義 */
#define RETRY_TMO        10

/* リトライ時の次の通信開始まで待つためのソフトウェアループ回数の定義 */
#define RETRY_WAIT_TIME 1000

/* 送信サイズ */
#define SEND_SIZE        3

/* サンプルコードのモード管理用定義 */
typedef enum
{
    IDLE = 0U,                  /* アイドル中 */
    BUSY,                       /* I2C 通信中 */
    INITIALIZE,                 /* 簡易 I2C FIT モジュールの初期設定 */
    DEVICE_A_WRITE,             /* デバイス A への書き込み */
    FINISH,                     /* 通信終了 */
    RETRY_WAIT_DEV_A_WR,        /* デバイス A への書き込みのリトライ待ち */
    ERROR                       /* エラー発生 */
} sample_mode_t;

/* サンプルコードのモード管理用変数 */
volatile uint8_t    sample_mode;

/* リトライ回数管理用変数 */
uint32_t            retry_cnt;

/* 送信回数管理用変数 */
uint8_t             send_num = 0;

void main(void);
void Callback_deviceA(void);

void main(void)
{
    rsci_iic_return_t   ret;                        /* API 関数の戻り値確認用 */
    volatile uint32_t  retry_wait_cnt = 0;         /* リトライ間隔調整用カウンタ */

    rsci_iic_info_t  iic_info_deviceA;              /* デバイス A 用情報構造体 */
    uint8_t slave_addr_deviceA[1]   = {0x50};      /* デバイス A 用スレーブアドレス */
    uint8_t access_addr_deviceA[1]  = {0x00};      /* デバイス A 用アクセス先アドレス */
    uint8_t send_data[6]            = {0x81,0x82,0x83,0x84,0x85,0x86};  /* 送信データ */
```

図 6.1　1 つのチャネルで 1 つのスレーブデバイスに連続アクセスする例(1)

```
sample_mode = INITIALIZE;                          /* 次に"初期設定"を行う */

while(1)
{
    switch(sample_mode)
    {
        /* アイドル中 */
        case IDLE:
            /* Nothing to do */
        break;

        /* I2C 通信中 */
        case BUSY:
            /* Nothing to do */
        break;

        /* 簡易 I2C FIT モジュールの初期設定 */
        case INITIALIZE:
            /* デバイス A へ初回の通信か */
            if (0 == send_num)
            {
                /* ch0 は通信中か */
                if (RSCI_IIC_COMMUNICATION == g_rsci_iic_ChStatus[0])
                {
                    sample_mode = ERROR;           /* 次に"エラー時の処理"を行う */
                }
                else
                {
                    /* デバイス A の情報構造体の設定(送信パターン 1) */
                    iic_info_deviceA.p_slv_adr = slave_addr_deviceA;
                    iic_info_deviceA.p_data1st = access_addr_deviceA;
                    iic_info_deviceA.p_data2nd = send_data;
                    iic_info_deviceA.dev_sts = RSCI_IIC_NO_INIT;
                    iic_info_deviceA.cnt1st = sizeof(access_addr_deviceA);
                    iic_info_deviceA.cnt2nd = SEND_SIZE;
                    iic_info_deviceA.callbackfunc = &Callback_deviceA;
                    iic_info_deviceA.ch_no = 0;
                }

                retry_cnt = 0;

                /* RSCI open */
                ret = R_RSCI_IIC_Open(&iic_info_deviceA);

                if (RSCI_IIC_SUCCESS == ret)
                {
                    sample_mode = DEVICE_A_WRITE;/* 次に"デバイス A への書き込み"を行う */
                }
                else
                {
                    /* R_RSCI_IIC_Open()関数コール時のエラー処理 */
                    sample_mode = ERROR;           /* 次に"エラー時の処理"を行う */
                }
            }
            /* デバイス A へ連続した 2 回目以降の通信か */
            else if (1 <= send_num)
            {
                /* ch0 は通信中か */
                if (RSCI_IIC_COMMUNICATION == g_rsci_iic_ChStatus[0])
                {
                    sample_mode = ERROR;           /* 次に"エラー時の処理"を行う */
                }
                else
                {
                    /* デバイス A の情報構造体の設定(マスタ送信パターン 1) */
                    access_addr_deviceA[0] = (access_addr_deviceA[0] + SEND_SIZE);
                    iic_info_deviceA.p_data1st = access_addr_deviceA;
                    iic_info_deviceA.p_data2nd = (send_data + (SEND_SIZE * send_num));
                    iic_info_deviceA.cnt1st = sizeof(access_addr_deviceA);
                    iic_info_deviceA.cnt2nd = SEND_SIZE;
```

アイドル中や I²C 通信中は、
空処理でループする

チャネルの状態をグローバル変数の
g_rsci_iic_ChStatus[ ]で確認できる

2 回目の送信を行うため、
送信カウンタやポインタを初期化する

図 6.2　1 つのチャネルで 1 つのスレーブデバイスに連続アクセスする例(2)

```
                sample_mode = DEVICE_A_WRITE; /* 次に"デバイス A への書き込み"を行う */
            }
        }
    break;

    /* デバイス A への書き込み */
    case DEVICE_A_WRITE:
        retry_cnt = retry_cnt + 1;

        /* マスタ送信の開始 */
        ret = R_RSCI_IIC_MasterSend(&iic_info_deviceA);

        if (RSCI_IIC_SUCCESS == ret)
        {
            sample_mode = BUSY;              /* 次に"I2C 通信中"となる */
        }
        else if (RSCI_IIC_ERR_BUS_BUSY == ret)
        {
            sample_mode = RETRY_WAIT_DEV_A_WR; /* 次に"リトライ待ち"を行う */
        }
        else
        {
            /* R_RSCI_IIC_MasterSend()関数コール時のエラー処理 */
            sample_mode = ERROR;             /* 次に"エラー時の処理"を行う */
        }
    break;

    /* デバイス A への書き込みのリトライ待ち */
    case RETRY_WAIT_DEV_A_WR:
        retry_wait_cnt = retry_wait_cnt + 1;

        if (RETRY_TMO < retry_cnt)
        {
            retry_wait_cnt = 0;
            sample_mode = ERROR;             /* 次に"エラー時の処理"を行う */
        }

        if (RETRY_WAIT_TIME < retry_wait_cnt)
        {
            retry_wait_cnt = 0;

            switch (sample_mode)
            {
                case RETRY_WAIT_DEV_A_WR:
                    sample_mode = DEVICE_A_WRITE; /* 次に"デバイス A への書き込み"を行う */
                break;

                default:
                    /* Nothing to do */
                break;
            }
        }
    break;
```

R_RSCI_IIC_MasterSend 関数実行により、ST 生成から SP 生成まで FIT モジュールで処理される
SP 出力後に設定したコールバック関数(Callback_deviceA())が呼び出される

通信相手デバイスが EEPROM の場合、書き込みコマンドを送信し書くと、EEPROM の書き込み処理が完了するまでは、NACK 応答となる
サンプルコードでは、ACK 応答になるまで通信開始のリトライ処理をしています

図 6.3　1 つのチャネルで 1 つのスレーブデバイスに連続アクセスする例(3)

```
            /* 通信終了 */
            case FINISH:
                /* RSCI close */
                ret = R_RSCI_IIC_Close(&iic_info_deviceA);

                if (RSCI_IIC_SUCCESS == ret)
                {
                    sample_mode = IDLE;                 /* 次に"アイドル中"となる */
                }
                else
                {
                    /* R_RSCI_IIC_Close()関数コール時のエラー処理 */
                    sample_mode = ERROR;                /* 次に"エラー時の処理"を行う */
                }
            break;

            /* エラー発生 */
            case ERROR:
                /* Nothing to do */
            break;

            default:
                /* Nothing to do */
            break;
        }
    }
}


void Callback_deviceA(void)
{
    volatile rsci_iic_return_t ret;
    rsci_iic_mcu_status_t iic_status;
    rsci_iic_info_t iic_info_ch;
    iic_info_ch.ch_no = 0;

    /* 簡易 I2C ステータスの取得 */
    ret = R_RSCI_IIC_GetStatus(&iic_info_ch, &iic_status);

    if (RSCI_IIC_SUCCESS != ret)
    {
        /* R_RSCI_IIC_GetStatus()関数コール時のエラー処理 */
        sample_mode = ERROR;                            /* 次に"エラー時の処理"を行う */
    }
    else
    {
        if (1 == iic_status.BIT.NACK)
        {
            /* iic_status のステータスフラグを確認して
               NACK が検出されていた場合の処理 */
            sample_mode = RETRY_WAIT_DEV_A_WR;
        }
        else
        {
            retry_cnt = 0;
            send_num++;
            if (1 >= send_num)
            {
                sample_mode = INITIALIZE;               /* 次に"初期設定"を行う */
            }
            else
            {
                sample_mode = FINISH;                   /* 次に"通信終了"を行う */
            }
        }
    }
}
```

通信が終了した場合に R_RSCI_IIC_Close 関数を呼び出すことで使用している RSCI チャネルを解放することができる
R_RSCI_IIC_Close 関数は主に次の場合に呼び出してください
・低消費電力モードに移行する場合
・通信異常が発生した場合
・使用している RSCI のチャネルを解放したい場合

図 6.4　1 つのチャネルで 1 つのスレーブデバイスに連続アクセスする例(4)

6.2　1 つのチャネルで 2 つのスレーブデバイスにアクセスする場合の例

RSCI の 1 つのチャネルを簡易 I²C モードで使用し、2 つのスレーブデバイスに対して書き込みおよび読み出しを行う場合のサンプルコードを示します。

サンプルコードでは、アクセスするデバイスごとに、I²C 通信情報構造体を用意しています。

次の(1)～(4)の順に動作します。

(1) RSCI の ch0 を簡易 I²C FIT モジュールで使用可能にするため、R_RSCI_IIC_Open 関数を実行する。

(2) デバイス A に 3 バイトのデータを書き込むため、R_RSCI_IIC_MasterSend 関数を実行する。

(3) デバイス B から 3 バイトのデータを読み出すため、R_RSCI_IIC_MasterReceive 関数を実行する。

(4) RSCI の ch0 を簡易 I²C FIT モジュールから解放するため、R_RSCI_IIC_Close 関数を実行する。

```
#include <stddef.h> // NULL definition
#include "platform.h"
#include "r_rsci_iic_rx_if.h"

/* NACK 検出時のリトライ回数の定義 */
#define RETRY_TMO        10

/* リトライ時の次の通信開始まで待つためのソフトウェアループ回数の定義 */
#define RETRY_WAIT_TIME 1000

/* 送信サイズ */
#define SEND_SIZE        3

/* 受信サイズ */
#define RECEIVE_SIZE     3

/* サンプルコードのモード管理用定義 */
typedef enum
{
    IDLE = 0U,                  /* アイドル中 */
    BUSY,                       /* I2C 通信中 */
    INITIALIZE,                 /* 簡易 I2C FIT モジュールの初期設定 */
    DEVICE_A_WRITE,             /* デバイス A への書き込み */
    DEVICE_B_READ,              /* デバイス B からの読み出し */
    FINISH,                     /* 通信終了 */
    RETRY_WAIT_DEV_A_WR,        /* デバイス A への書き込みのリトライ待ち */
    RETRY_WAIT_DEV_B_RD,        /* デバイス B からの読み出しのリトライ待ち */
    ERROR                       /* エラー発生 */
} sample_mode_t;

/* サンプルコードのモード管理用変数 */
volatile uint8_t                sample_mode;

/* リトライ回数管理用変数 */
volatile uint32_t               retry_cnt;

void main(void);
void Callback_deviceA(void);
void Callback_deviceB(void);

void main(void)
{
    volatile rsci_iic_return_t   ret;                /* API 関数の戻り値確認用 */
    volatile uint32_t            retry_wait_cnt = 0; /* リトライ間隔調整用カウンタ */

    rsci_iic_info_t  iic_info_deviceA;               /* デバイス A 用情報構造体 */
    rsci_iic_info_t  iic_info_deviceB;               /* デバイス B 用情報構造体 */
```

プログラムの説明で、
次の略称を使用しています。
ST:スタートコンディション
SP:ストップコンディション

通信するデバイスの数だけ、
情報構造体を宣言する

図 6.5　1 つのチャネルで 2 つのスレーブデバイスにアクセスする例(1)

```
uint8_t slave_addr_deviceA[1]   = {0x51};        /* デバイス A 用スレーブアドレス */
uint8_t slave_addr_deviceB[1]   = {0x52};        /* デバイス B 用スレーブアドレス */
uint8_t access_addr_deviceA[1]  = {0x00};        /* デバイス A 用アクセス先アドレス */
uint8_t access_addr_deviceB[2]  = {0x00,0x00};   /* デバイス B 用アクセス先アドレス */
uint8_t send_data[5]            = {0x81,0x82,0x83,0x84,0x85}; /* 送信データ */
uint8_t store_area[5]           = {0xFF,0xFF,0xFF,0xFF,0xFF}; /* 受信データ格納用 */

sample_mode = INITIALIZE;                        /* 次に"初期設定"を行う */

while(1)
{
    switch(sample_mode)
    {
        /* アイドル中 */
        case IDLE:
            /* Nothing to do */
        break;

        /* I2C 通信中 */
        case BUSY:
            /* Nothing to do */
        break;

        /* 簡易 I2C FIT モジュールの初期設定 */
        case INITIALIZE:
            /* チャネル 0 は通信中か */
            if (RSCI_IIC_COMMUNICATION == g_rsci_iic_ChStatus[0])
            {
                sample_mode = ERROR;             /* 次に"エラー時の処理"を行う */
            }
            else
            {
                /* デバイス A の情報構造体の設定(マスタ送信パターン 1) */
                iic_info_deviceA.p_slv_adr = slave_addr_deviceA;
                iic_info_deviceA.p_data1st = access_addr_deviceA;
                iic_info_deviceA.p_data2nd = send_data;
                iic_info_deviceA.dev_sts = RSCI_IIC_NO_INIT;
                iic_info_deviceA.cnt1st = sizeof(access_addr_deviceA);
                iic_info_deviceA.cnt2nd = SEND_SIZE;
                iic_info_deviceA.callbackfunc = &Callback_deviceA;
                iic_info_deviceA.ch_no = 0;

                /* デバイス B の情報構造体の設定(マスタ送受信) */
                iic_info_deviceB.p_slv_adr = slave_addr_deviceB;
                iic_info_deviceB.p_data1st = access_addr_deviceB;
                iic_info_deviceB.p_data2nd = store_area;
                iic_info_deviceB.dev_sts = RSCI_IIC_NO_INIT;
                iic_info_deviceB.cnt1st = sizeof(access_addr_deviceB);
                iic_info_deviceB.cnt2nd = RECEIVE_SIZE;
                iic_info_deviceB.callbackfunc = &Callback_deviceB;
                iic_info_deviceB.ch_no = 0;
            }

            retry_cnt = 0;                       /* リトライ回数の初期化 */

            /* RSCI open */
            ret = R_RSCI_IIC_Open(&iic_info_deviceA);

            if (RSCI_IIC_SUCCESS == ret)
            {
                sample_mode = DEVICE_A_WRITE;    /* 次に"デバイス A への書き込み"を行う */
            }
            else
            {
                /* R_RSCI_IIC_Open()関数コール時のエラー処理 */
                sample_mode = ERROR;             /* 次に"エラー時の処理"を行う */
            }

        break;
```

アイドル中や I²C 通信中は、
空処理でループする

チャネルの状態をグローバル変数の
g_rsci_iic_ChStatus[ ]で確認できる

RSCI リソースの確保は、チャネルごとに行うため、
R_RSCI_IIC_Open 関数は 1 回だけ実行する

図 6.6　1 つのチャネルで 2 つのスレーブデバイスにアクセスする例(2)

R_RSCI_IIC_MasterSend 関数実行により、
ST 生成から SP 生成まで FIT モジュールで処理される
SP 出力後に設定したコールバック関数
(Callback_deviceA())が呼び出される

```
/* デバイス A への書き込み */
case DEVICE_A_WRITE:
    retry_cnt = retry_cnt + 1;

    /* マスタ送信の開始 */
    ret = R_RSCI_IIC_MasterSend(&iic_info_deviceA);

    if (RSCI_IIC_SUCCESS == ret)
    {
        sample_mode = BUSY;              /* 次に"I2C 通信中"となる */
    }
    else if (RSCI_IIC_ERR_BUS_BUSY == ret)
    {
        sample_mode = RETRY_WAIT_DEV_A_WR; /* 次に"リトライ待ち"を行う */
    }
    else
    {
        /* R_RSCI_IIC_MasterSend()関数コール時のエラー処理 */
        sample_mode = ERROR;             /* 次に"エラー時の処理"を行う */
    }
break;
```

R_RSCI_IIC_MasterReceive 関数実行により、
ST 生成から SP 生成まで FIT モジュールで処理される
SP 出力後に設定したコールバック関数
(Callback_deviceB())が呼び出される

```
/* デバイス B からの読み出し */
case DEVICE_B_READ:
    retry_cnt = retry_cnt + 1;

    /* マスタ送受信の開始 */
    ret = R_RSCI_IIC_MasterReceive(&iic_info_deviceB);

    if (RSCI_IIC_SUCCESS == ret)
    {
        sample_mode = BUSY;               /* 次に"I2C 通信中"となる */
    }
    else if (RSCI_IIC_ERR_BUS_BUSY == ret)
    {
        sample_mode = RETRY_WAIT_DEV_B_RD; /* 次に"リトライ待ち"を行う */
    }
    else
    {
        /* R_RSCI_IIC_MasterReceive()関数コール時のエラー処理 */
        sample_mode = ERROR;             /* 次に"エラー時の処理"を行う */
    }
break;
```

通信相手デバイスが EEPROM の場合、
書き込みコマンドを送信し書くと、
EEPROM の書き込み処理が完了するまでは、
NACK 応答となる
サンプルコードでは、ACK 応答になるまで
通信開始のリトライ処理をしています

```
/* デバイス A への書き込みのリトライ待ち */
/* デバイス B からの読み出しのリトライ待ち */
case RETRY_WAIT_DEV_A_WR:
case RETRY_WAIT_DEV_B_RD:
    retry_wait_cnt = retry_wait_cnt + 1;

    if (RETRY_TMO < retry_cnt)
    {
        retry_wait_cnt = 0;
        sample_mode = ERROR;             /* 次に"エラー時の処理"を行う */
    }

    if (RETRY_WAIT_TIME < retry_wait_cnt)
    {
        retry_wait_cnt = 0;

        switch (sample_mode)
        {
            case RETRY_WAIT_DEV_A_WR:
                sample_mode = DEVICE_A_WRITE; /* 次に"デバイス A への書き込み"を行う */
            break;

            case RETRY_WAIT_DEV_B_RD:
                sample_mode = DEVICE_B_READ; /* 次に"デバイス B からの読み出し"を行う */
            break;
```

図 6.7　1 つのチャネルで 2 つのスレーブデバイスにアクセスする例(3)

```
                        default:
                            /* Nothing to do */
                        break;
                    }
                }
            break;

            /* 通信終了 */
            case FINISH:
                /* RSCI close */
                ret = R_RSCI_IIC_Close(&iic_info_deviceA);

                if (RSCI_IIC_SUCCESS == ret)
                {
                    sample_mode = IDLE;                 /* 次に"アイドル中"となる */
                }
                else
                {
                    /* R_RSCI_IIC_Close()関数コール時のエラー処理 */
                    sample_mode = ERROR;                /* 次に"エラー時の処理"を行う */
                }
            break;

            /* エラー発生 */
            case ERROR:
                /* Nothing to do */
            break;

            default:
                /* Nothing to do */
            break;
        }
    }
}

void Callback_deviceA(void)
{
    volatile rsci_iic_return_t ret;
    rsci_iic_mcu_status_t iic_status;
    rsci_iic_info_t iic_info_ch;
    iic_info_ch.ch_no = 0;

    /* 簡易 I2C ステータスの取得 */
    ret = R_RSCI_IIC_GetStatus(&iic_info_ch, &iic_status);

    if (RSCI_IIC_SUCCESS != ret)
    {
        /* R_RSCI_IIC_GetStatus()関数コール時のエラー処理 */
        sample_mode = ERROR;                            /* 次に"エラー時の処理"を行う */
    }
    else
    {
        if (1 == iic_status.BIT.NACK)
        {
            /* iic_status のステータスフラグを確認して
               NACK が検出されていた場合の処理 */
            sample_mode = RETRY_WAIT_DEV_A_WR;          /* 次に"リトライ待ち"を行う */
        }
        else
        {
            retry_cnt = 0;
            sample_mode = DEVICE_B_READ;                /* 次に"デバイス B からの読み出し"を行う */
        }
    }
}

void Callback_deviceB(void)
{
    volatile rsci_iic_return_t ret;
    rsci_iic_mcu_status_t iic_status;
    rsci_iic_info_t iic_info_ch;
    iic_info_ch.ch_no = 0;
```

通信が終了した場合に R_RSCI_IIC_Close 関数を呼び出すことで使用している RSCI チャネルを解放することができる
R_RSCI_IIC_Close 関数は主に次の場合に呼び出してください
・低消費電力モードに移行する場合
・通信異常が発生した場合
・使用している RSCI のチャネルを解放したい場合

図 6.8　1 つのチャネルで 2 つのスレーブデバイスにアクセスする例(4)

```
    /* 簡易 I2C ステータスの取得 */
    ret = R_RSCI_IIC_GetStatus(&iic_info_ch, &iic_status);

    if (RSCI_IIC_SUCCESS != ret)
    {
        /* R_RSCI_IIC_GetStatus()関数コール時のエラー処理 */
        sample_mode = ERROR;                        /* 次に"エラー時の処理"を行う */
    }
    else
    {
        if (1 == iic_status.BIT.NACK)
        {
            /* iic_status のステータスフラグを確認して
               NACK が検出されていた場合の処理 */
            sample_mode = RETRY_WAIT_DEV_B_RD;      /* 次に"リトライ待ち"を行う */
        }
        else
        {
            retry_cnt = 0;
            sample_mode = FINISH;                   /* 次に"通信終了"を行う */
        }

    }
}
```

図 6.9　1 つのチャネルで 2 つのスレーブデバイスにアクセスする例(5)

6.3　2 つのチャネルで 2 つのスレーブデバイスにアクセスする場合の例

RSCI の 2 つのチャネルを簡易 I²C モードで使用し、各チャネルはそれぞれ異なるスレーブデバイスに対して書き込みおよび読み出しを行う場合のサンプルコードを示します。

サンプルコードでは、アクセスするデバイスごとに、I²C 通信情報構造体を用意しています。

次の(1)～(3)の順に動作します。

(1) RSCI の ch0 を簡易 I²C FIT モジュールで使用可能にするため、R_RSCI_IIC_Open 関数を実行する。RSCI の ch8 を簡易 I²C FIT モジュールで使用可能にするため、R_RSCI_IIC_Open 関数を実行する。

(2) RSCI の ch0 を使用し、デバイス A に 3 バイトのデータを書き込むため、 R_RSCI_IIC_MasterSend 関数を実行する。 RSCI の ch8 を使用し、デバイス B から 3 バイトのデータを読み出すため、R_RSCI_IIC_MasterReceive 関数を実行する。

(3) RSCI の ch0 を簡易 I²C FIT モジュールから解放するため、R_RSCI_IIC_Close 関数を実行する。RSCI の ch8 を簡易 I²C FIT モジュールから解放するため、R_RSCI_IIC_Close 関数を実行する。

プログラムの説明で、
次の略称を使用しています。
ST:スタートコンディション
SP:ストップコンディション

```
#include <stddef.h> /* NULL definition */
#include "platform.h"
#include "r_rsci_iic_rx_if.h"

/* NACK 検出時のリトライ回数の定義 */
#define RETRY_TMO        10

/* リトライ時の次の通信開始まで待つためのソフトウェアループ回数の定義 */
#define RETRY_WAIT_TIME 1000

/* 送信サイズ */
#define SEND_SIZE        3

/* 受信サイズ */
#define RECEIVE_SIZE     3

/* サンプルコードのモード管理用定義 */
typedef enum
{
    IDLE = 0U,                  /* アイドル中 */
    BUSY,                       /* I2C 通信中 */
    INITIALIZE,                 /* 簡易 I2C FIT モジュールの初期設定 */
    DEVICE_A_WRITE,             /* デバイス A への書き込み */
    DEVICE_B_READ,              /* デバイス B からの読み出し */
    FINISH,                     /* 通信終了 */
    RETRY_WAIT_DEV_A_WR,        /* デバイス A への書き込みのリトライ待ち */
    RETRY_WAIT_DEV_B_RD,        /* デバイス B からの読み出しのリトライ待ち */
    ERROR                       /* エラー発生 */
} sample_mode_t;

/* サンプルコードのモード管理用変数 */
volatile uint8_t                sample_mode_ch0;
volatile uint8_t                sample_mode_ch8;

/* リトライ回数管理用変数 */
volatile uint32_t               retry_cnt_ch0;
volatile uint32_t               retry_cnt_ch8;

void main(void);
void Callback_deviceA(void);
void Callback_deviceB(void);

void main(void)
{
```

図 6.10　2 つのチャネルで 2 つのスレーブデバイスにアクセスする例(1)

```
volatile rsci_iic_return_t   ret;                    /* API 関数の戻り値確認用 */
volatile uint32_t retry_wait_cnt_ch0 = 0;           /* リトライ間隔調整用カウンタ */
volatile uint32_t retry_wait_cnt_ch8 = 0;           /* リトライ間隔調整用カウンタ */

rsci_iic_info_t  iic_info_deviceA;                   /* デバイス A 用情報構造体 */
rsci_iic_info_t  iic_info_deviceB;                   /* デバイス B 用情報構造体 */
uint8_t slave_addr_deviceA[1]   = {0x50};           /* デバイス A 用スレーブアドレス */
uint8_t slave_addr_deviceB[1]   = {0x50};           /* デバイス B 用スレーブアドレス */
uint8_t access_addr_deviceA[1]  = {0x00};           /* デバイス A 用アクセス先アドレス */
uint8_t access_addr_deviceB[2]  = {0x00,0x00};      /* デバイス B 用アクセス先アドレス */
uint8_t send_data[5]            = {0x81,0x82,0x83,0x84,0x85}; /* 送信データ */
uint8_t store_area[5]           = {0xFF,0xFF,0xFF,0xFF,0xFF}; /* 受信データ格納用 */

sample_mode_ch0 = INITIALIZE;                       /* ch0 は、次に"初期設定"を行う */
sample_mode_ch8 = INITIALIZE;                       /* ch8 は、次に"初期設定"を行う */

while(1)
{
    switch(sample_mode_ch0)
    {
        /* アイドル中 */
        case IDLE:
            /* Nothing to do */
        break;

        /* I2C 通信中 */
        case BUSY:
            /* Nothing to do */
        break;

        /* 簡易 I2C FIT モジュールの初期設定 */
        case INITIALIZE:
            /* チャネル 0 は通信中か */
            if (RSCI_IIC_COMMUNICATION == g_rsci_iic_ChStatus[0])
            {
                sample_mode_ch0 = ERROR;            /* ch0 は、次に"エラー時の処理"を行う */
            }
            else
            {
                /* デバイス A の情報構造体の設定(マスタ送信パターン 1) */
                iic_info_deviceA.p_slv_adr = slave_addr_deviceA;
                iic_info_deviceA.p_data1st = access_addr_deviceA;
                iic_info_deviceA.p_data2nd = send_data;
                iic_info_deviceA.dev_sts = RSCI_IIC_NO_INIT;
                iic_info_deviceA.cnt1st = sizeof(access_addr_deviceA);
                iic_info_deviceA.cnt2nd = SEND_SIZE;
                iic_info_deviceA.callbackfunc = &Callback_deviceA;
                iic_info_deviceA.ch_no = 0;
            }

            retry_cnt_ch0 = 0;                      /* リトライ回数の初期化 */

            /* RSCI open */
            ret = R_RSCI_IIC_Open(&iic_info_deviceA);

            if (RSCI_IIC_SUCCESS == ret)
            {
                sample_mode_ch0 = DEVICE_A_WRITE; /* ch0 は、次に"デバイス A への書き込み"を行う */
            }
            else
            {
                /* R_RSCI_IIC_Open()関数コール時のエラー処理 */
                sample_mode_ch0 = ERROR;            /* ch0 は、次に"エラー時の処理"を行う */
            }
        break;

        /* デバイス A への書き込み */
        case DEVICE_A_WRITE:
            retry_cnt_ch0 = retry_cnt_ch0 + 1;
```

アクセスするデバイスごとに
情報構造体を用意する

異なるチャネルはそれぞれ同時に動作できるため
チャネルごとにモードを管理する

アイドル中や I²C 通信中は、
空処理でループする

チャネルの状態をグローバル変数の
g_rsci_iic_ChStatus[ ]で確認できる

図 6.11　2 つのチャネルで 2 つのスレーブデバイスにアクセスする例(2)

```
            /* マスタ送信の開始 */
            ret = R_RSCI_IIC_MasterSend(&iic_info_deviceA);

            if (RSCI_IIC_SUCCESS == ret)
            {
                sample_mode_ch0 = BUSY;          /* ch0 は、次に"I2C 通信中"となる */
            }
            else if (RSCI_IIC_ERR_BUS_BUSY == ret)
            {
                sample_mode_ch0 = RETRY_WAIT_DEV_A_WR;  /* ch0 は、次に"リトライ待ち"を行う */
            }
            else
            {
                /* R_RSCI_IIC_MasterSend()関数コール時のエラー処理 */
                sample_mode_ch0 = ERROR;         /* ch0 は、次に"エラー時の処理"を行う */
            }
        break;
```

この関数実行により、ST 生成から SP 生成まで FIT モジュールで処理される
SP 出力後に設定したコールバック関数(Callback_deviceA())が呼び出される

```
        /* デバイス A への書き込みのリトライ待ち */
        case RETRY_WAIT_DEV_A_WR:
            retry_wait_cnt_ch0 = retry_wait_cnt_ch0 + 1;

            if (RETRY_TMO < retry_cnt_ch0)
            {
                retry_wait_cnt_ch0 = 0;
                sample_mode_ch0 = ERROR;         /* ch0 は、次に"エラー時の処理"を行う */
            }

            if (RETRY_WAIT_TIME < retry_wait_cnt_ch0)
            {
                retry_wait_cnt_ch0 = 0;

                switch (sample_mode_ch0)
                {
                    case RETRY_WAIT_DEV_A_WR:
                        sample_mode_ch0 = DEVICE_A_WRITE;   /* ch0 は、次に"デバイス A への書き込
み"を行う */

                    break;

                    default:
                        /* Nothing to do */
                    break;
                }
            }
        break;
```

通信相手デバイスが EEPROM の場合、
書き込みコマンドを送信し書くと、
EEPROM の書き込み処理が
完了するまでは、NACK 応答となる
サンプルコードでは、ACK 応答になるまで
通信開始のリトライ処理をしています

```
        /* 通信終了 */
        case FINISH:
            /* RSCI close */
            ret = R_RSCI_IIC_Close(&iic_info_deviceA)

            if (RSCI_IIC_SUCCESS == ret)
            {
                sample_mode_ch0 = IDLE;          /* ch0 は、次に"アイドル中"となる */
            }
            else
            {
                /* R_RSCI_IIC_Close()関数コール時のエラー処理 */
                sample_mode_ch0 = ERROR;         /* ch0 は、次に"エラー時の処理"を行う */
            }
        break;

        /* エラー発生 */
        case ERROR:
            /* Nothing to do */
        break;

        default:
            /* Nothing to do */
        break;
    }
```

通信が終了した場合に R_RSCI_IIC_Close 関数を呼び出すことで
使用している RSCI チャネルを解放することができる
R_RSCI_IIC_Close 関数は主に次の場合に呼び出してください
・低消費電力モードに移行する場合
・通信異常が発生した場合
・使用している RSCI のチャネルを解放したい場合

図 6.12　2 つのチャネルで 2 つのスレーブデバイスにアクセスする例(3)

```
switch(sample_mode_ch8)
{
    /* アイドル中 */
    case IDLE:
        /* Nothing to do */
    break;

    /* I2C 通信中 */
    case BUSY:
        /* Nothing to do */
    break;

    /* 簡易 I2C FIT モジュールの初期設定 */
    case INITIALIZE:
        /* チャネル 8 は通信中か */
        if (RSCI_IIC_COMMUNICATION == g_rsci_iic_ChStatus[8])
        {
            sample_mode_ch8 = ERROR;        /* ch8 は、次に"エラー時の処理"を行う */
        }
        else
        {
            /* デバイス B の情報構造体の設定(マスタ送受信) */
            iic_info_deviceB.p_slv_adr = slave_addr_deviceB;
            iic_info_deviceB.p_data1st = access_addr_deviceB;
            iic_info_deviceB.p_data2nd = store_area;
            iic_info_deviceB.dev_sts = RSCI_IIC_NO_INIT;
            iic_info_deviceB.cnt1st = sizeof(access_addr_deviceB);
            iic_info_deviceB.cnt2nd = RECEIVE_SIZE;
            iic_info_deviceB.callbackfunc = &Callback_deviceB;
            iic_info_deviceB.ch_no = 8;
        }

        retry_cnt_ch8 = 0;                  /* リトライ回数の初期化 */

        /* RSCI open */
        ret = R_RSCI_IIC_Open(&iic_info_deviceB);

        if (RSCI_IIC_SUCCESS == ret)
        {
            sample_mode_ch8 = DEVICE_B_READ; /* ch8 は、次に"デバイス B からの読み出し"を行う */
        }
        else
        {
            /* R_RSCI_IIC_Open()関数コール時のエラー処理 */
            sample_mode_ch8 = ERROR;        /* ch8 は、次に"エラー時の処理"を行う */
        }

    break;

    case DEVICE_B_READ:
        retry_cnt_ch8 = retry_cnt_ch8 + 1;

        /* マスタ送受信の開始 */
        ret = R_RSCI_IIC_MasterReceive(&iic_info_deviceB);

        if (RSCI_IIC_SUCCESS == ret)
        {
            sample_mode_ch8 = BUSY;         /* ch8 は、次に"I2C 通信中"となる */
        }
        else if (RSCI_IIC_ERR_BUS_BUSY == ret)
        {
            sample_mode_ch8 = RETRY_WAIT_DEV_B_RD;  /* ch8 は、次に"リトライ待ち"を行う */
        }
        else
        {
            /* R_RSCI_IIC_MasterReceive()関数コール時のエラー処理 */
            sample_mode_ch8 = ERROR;        /* ch8 は、次に"エラー時の処理"を行う */
        }
    break;
```

アイドル中や I²C 通信中は、空処理でループする

チャネルの状態をグローバル変数の g_rsci_iic_ChStatus[ ]で確認できる

この関数実行により、ST 生成から SP 生成まで FIT モジュールで処理される
SP 出力後に設定したコールバック関数(Callback_deviceB())が呼び出される

図 6.13　2 つのチャネルで 2 つのスレーブデバイスにアクセスする例(4)

```
            /* デバイス B からの読み出しのリトライ待ち */
            case RETRY_WAIT_DEV_B_RD:
                retry_wait_cnt_ch8 = retry_wait_cnt_ch8 + 1;

                if (RETRY_TMO < retry_cnt_ch8)
                {
                    retry_wait_cnt_ch8 = 0;
                    sample_mode_ch8 = ERROR;        /* ch8 は、次に"エラー時の処理"を行う */
                }

                if (RETRY_WAIT_TIME < retry_wait_cnt_ch8)
                {
                    retry_wait_cnt_ch8 = 0;

                    switch (sample_mode_ch8)
                    {
                        case RETRY_WAIT_DEV_B_RD:
                            sample_mode_ch8 = DEVICE_B_READ;    /* ch8 は、次に"デバイス B からの読み出
し"を行う */

                        break;

                        default:
                            /* Nothing to do */
                        break;
                    }
                }
            break;

            /* 通信終了 */
            case FINISH:
                /* RSCI close */
                ret = R_RSCI_IIC_Close(&iic_info_deviceB);

                if (RSCI_IIC_SUCCESS == ret)
                {
                    sample_mode_ch8 = IDLE;         /* ch8 は、次に"アイドル中"となる */
                }
                else
                {
                    /* R_RSCI_IIC_Close()関数コール時のエラー処理 */
                    sample_mode_ch8 = ERROR;        /* ch8 は、次に"エラー時の処理"を行う */
                }
            break;

            /* エラー発生 */
            case ERROR:
                /* Nothing to do */
            break;

            default:
                /* Nothing to do */
            break;
        }

    }
}

void Callback_deviceA(void)
{
    volatile rsci_iic_return_t ret;
    rsci_iic_mcu_status_t iic_status;
    rsci_iic_info_t iic_info_ch;
    iic_info_ch.ch_no = 0;

    /* 簡易 I2C ステータスの取得 */
    ret = R_RSCI_IIC_GetStatus(&iic_info_ch, &iic_status);

    if (RSCI_IIC_SUCCESS != ret)
    {
        /* R_RSCI_IIC_GetStatus()関数コール時のエラー処理 */
        sample_mode_ch0 = ERROR;                    /* ch0 は、次に"エラー時の処理"を行う */
    }
```

通信相手デバイスが EEPROM の場合、
書き込みコマンドを送信し書くと、
EEPROM の書き込み処理が
完了するまでは、NACK 応答となる
サンプルコードでは、ACK 応答になるまで
通信開始のリトライ処理をしています

通信が終了した場合に R_RSCI_IIC_Close 関数を呼び出すことで
使用している RSCI チャネルを解放することができる
R_RSCI_IIC_Close 関数は主に次の場合に呼び出してください
・低消費電力モードに移行する場合
・通信異常が発生した場合
・使用している RSCI のチャネルを解放したい場合

図 6.14　2 つのチャネルで 2 つのスレーブデバイスにアクセスする例(5)

```
    else
    {
        if (1 == iic_status.BIT.NACK)
        {
            /* iic_status のステータスフラグを確認して
               NACK が検出されていた場合の処理 */
            sample_mode_ch0 = RETRY_WAIT_DEV_A_WR;  /* ch0 は、次に"リトライ待ち"を行う */
        }
        else
        {
            retry_cnt_ch0 = 0;
            sample_mode_ch0 = FINISH;               /* ch0 は、次に"通信終了"を行う */
        }
    }
}

void Callback_deviceB(void)
{
    volatile rsci_iic_return_t ret;
    rsci_iic_mcu_status_t iic_status;
    rsci_iic_info_t iic_info_ch;
    iic_info_ch.ch_no = 8;

    /* 簡易 I2C ステータスの取得 */
    ret = R_RSCI_IIC_GetStatus(&iic_info_ch, &iic_status);

    if (RSCI_IIC_SUCCESS != ret)
    {
        /* R_RSCI_IIC_GetStatus()関数コール時のエラー処理 */
        sample_mode_ch8 = ERROR;                    /* ch8 は、次に"エラー時の処理"を行う */
    }
    else
    {
        if (1 == iic_status.BIT.NACK)
        {
            /* iic_status のステータスフラグを確認して
               NACK が検出されていた場合の処理 */
            sample_mode_ch8 = RETRY_WAIT_DEV_B_RD;  /* ch8 は、次に"リトライ待ち"を行う */
        }
        else
        {
            retry_cnt_ch8 = 0;
            sample_mode_ch8 = FINISH;               /* ch8 は、次に"通信終了"を行う */
        }

    }
}
```

図 6.15　2 つのチャネルで 2 つのスレーブデバイスにアクセスする例(6)

7. 参考ドキュメント

ユーザーズマニュアル：ハードウェア

(最新版をルネサス エレクトロニクスホームページから入手してください。)

テクニカルアップデート／テクニカルニュース

(最新の情報をルネサス エレクトロニクスホームページから入手してください。)

ユーザーズマニュアル：開発環境

RX ファミリ C/C++コンパイラ CC-RX ユーザーズマニュアル (R20UT3248)

(最新版をルネサス エレクトロニクスホームページから入手してください。)

テクニカルアップデートの対応について

本モジュールは以下のテクニカルアップデートの内容を反映しています。

- なし

改訂記録

Rev.	発行日	改訂内容	
		ページ	ポイント
1.00	2025.11.28	—	初版発行。

製品ご使用上の注意事項

ここでは、マイコン製品全体に適用する「使用上の注意事項」について説明します。個別の使用上の注意事項については、本ドキュメントおよびテクニカルアップデートを参照してください。

1. 静電気対策
   CMOS 製品の取り扱いの際は静電気防止を心がけてください。CMOS 製品は強い静電気によってゲート絶縁破壊を生じることがあります。運搬や保存の際には、当社が出荷梱包に使用している導電性のトレーやマガジンケース、導電性の緩衝材、金属ケースなどを利用し、組み立て工程にはアースを施してください。プラスチック板上に放置したり、端子を触ったりしないでください。また、CMOS 製品を実装したボードについても同様の扱いをしてください。
2. 電源投入時の処置
   電源投入時は、製品の状態は不定です。電源投入時には、LSI の内部回路の状態は不確定であり、レジスタの設定や各端子の状態は不定です。外部リセット端子でリセットする製品の場合、電源投入からリセットが有効になるまでの期間、端子の状態は保証できません。同様に、内蔵パワーオンリセット機能を使用してリセットする製品の場合、電源投入からリセットのかかる一定電圧に達するまでの期間、端子の状態は保証できません。
3. 電源オフ時における入力信号
   当該製品の電源がオフ状態のときに、入力信号や入出力プルアップ電源を入れないでください。入力信号や入出力プルアップ電源からの電流注入により、誤動作を引き起こしたり、異常電流が流れ内部素子を劣化させたりする場合があります。資料中に「電源オフ時における入力信号」についての記載のある製品は、その内容を守ってください。
4. 未使用端子の処理
   未使用端子は、「未使用端子の処理」に従って処理してください。CMOS 製品の入力端子のインピーダンスは、一般に、ハイインピーダンスとなっています。未使用端子を開放状態で動作させると、誘導現象により、LSI 周辺のノイズが印加され、LSI 内部で貫通電流が流れたり、入力信号と認識されて誤動作を起こす恐れがあります。
5. クロックについて
   リセット時は、クロックが安定した後、リセットを解除してください。プログラム実行中のクロック切り替え時は、切り替え先クロックが安定した後に切り替えてください。リセット時、外部発振子（または外部発振回路）を用いたクロックで動作を開始するシステムでは、クロックが十分安定した後、リセットを解除してください。また、プログラムの途中で外部発振子（または外部発振回路）を用いたクロックに切り替える場合は、切り替え先のクロックが十分安定してから切り替えてください。
6. 入力端子の印加波形
   入力ノイズや反射波による波形歪みは誤動作の原因になりますので注意してください。CMOS 製品の入力がノイズなどに起因して、V_{IL}（Max.）から V_{IH}（Min.）までの領域にとどまるような場合は、誤動作を引き起こす恐れがあります。入力レベルが固定の場合はもちろん、V_{IL}（Max.）から V_{IH}（Min.）までの領域を通過する遷移期間中にチャタリングノイズなどが入らないように使用してください。
7. リザーブアドレス（予約領域）のアクセス禁止
   リザーブアドレス（予約領域）のアクセスを禁止します。アドレス領域には、将来の拡張機能用に割り付けられている リザーブアドレス（予約領域）があります。これらのアドレスをアクセスしたときの動作については、保証できませんので、アクセスしないようにしてください。
8. 製品間の相違について
   型名の異なる製品に変更する場合は、製品型名ごとにシステム評価試験を実施してください。同じグループのマイコンでも型名が違うと、フラッシュメモリ、レイアウトパターンの相違などにより、電気的特性の範囲で、特性値、動作マージン、ノイズ耐量、ノイズ輻射量などが異なる場合があります。型名が違う製品に変更する場合は、個々の製品ごとにシステム評価試験を実施してください。

ご注意書き

1. 本資料に記載された回路、ソフトウェアおよびこれらに関連する情報は、半導体製品の動作例、応用例を説明するものです。回路、ソフトウェアおよびこれらに関連する情報を使用する場合、お客様の責任において、お客様の機器・システムを設計ください。これらの使用に起因して生じた損害（お客様または第三者いずれに生じた損害も含みます。以下同じです。）に関し、当社は、一切その責任を負いません。
2. 当社製品または本資料に記載された製品データ、図、表、プログラム、アルゴリズム、応用回路例等の情報の使用に起因して発生した第三者の特許権、著作権その他の知的財産権に対する侵害またはこれらに関する紛争について、当社は、何らの保証を行うものではなく、また責任を負うものではありません。
3. 当社は、本資料に基づき当社または第三者の特許権、著作権その他の知的財産権を何ら許諾するものではありません。
4. 当社製品を組み込んだ製品の輸出入、製造、販売、利用、配布その他の行為を行うにあたり、第三者保有の技術の利用に関するライセンスが必要となる場合、当該ライセンス取得の判断および取得はお客様の責任において行ってください。
5. 当社製品を、全部または一部を問わず、改造、改変、複製、リバースエンジニアリング、その他、不適切に使用しないでください。かかる改造、改変、複製、リバースエンジニアリング等により生じた損害に関し、当社は、一切その責任を負いません。
6. 当社は、当社製品の品質水準を「標準水準」および「高品質水準」に分類しており、各品質水準は、以下に示す用途に製品が使用されることを意図しております。

   標準水準：　コンピュータ、OA 機器、通信機器、計測機器、AV 機器、家電、工作機械、パーソナル機器、産業用ロボット等

   高品質水準：輸送機器（自動車、電車、船舶等）、交通制御（信号）、大規模通信機器、金融端末基幹システム、各種安全制御装置等

   当社製品は、データシート等により高信頼性、Harsh environment 向け製品と定義しているものを除き、直接生命・身体に危害を及ぼす可能性のある機器・システム（生命維持装置、人体に埋め込み使用するもの等）、もしくは多大な物的損害を発生させるおそれのある機器・システム（宇宙機器と、海底中継器、原子力制御システム、航空機制御システム、プラント基幹システム、軍事機器等）に使用されることを意図しておらず、これらの用途に使用することは想定していません。たとえ、当社が想定していない用途に当社製品を使用したことにより損害が生じても、当社は一切その責任を負いません。
7. あらゆる半導体製品は、外部攻撃からの安全性を 100%保証されているわけではありません。当社ハードウェア／ソフトウェア製品にはセキュリティ対策が組み込まれているものもありますが、これによって、当社は、セキュリティ脆弱性または侵害（当社製品または当社製品が使用されているシステムに対する不正アクセス・不正使用を含みますが、これに限りません。）から生じる責任を負うものではありません。当社は、当社製品または当社製品が使用されたあらゆるシステムが、不正な改変、攻撃、ウイルス、干渉、ハッキング、データの破壊または窃盗その他の不正な侵入行為（「脆弱性問題」といいます。）によって影響を受けないことを保証しません。当社は、脆弱性問題に起因しまたはこれに関連して生じた損害について、一切責任を負いません。また、法令において認められる限りにおいて、本資料および当社ハードウェア／ソフトウェア製品について、商品性および特定目的との合致に関する保証ならびに第三者の権利を侵害しないことの保証を含め、明示または黙示のいかなる保証も行いません。
8. 当社製品をご使用の際は、最新の製品情報（データシート、ユーザーズマニュアル、アプリケーションノート、信頼性ハンドブックに記載の「半導体デバイスの使用上の一般的な注意事項」等）をご確認の上、当社が指定する最大定格、動作電源電圧範囲、放熱特性、実装条件その他指定条件の範囲内でご使用ください。指定条件の範囲を超えて当社製品をご使用された場合の故障、誤動作の不具合および事故につきましては、当社は、一切その責任を負いません。
9. 当社は、当社製品の品質および信頼性の向上に努めていますが、半導体製品はある確率で故障が発生したり、使用条件によっては誤動作したりする場合があります。また、当社製品は、データシート等において高信頼性、Harsh environment 向け製品と定義しているものを除き、耐放射線設計を行っておりません。仮に当社製品の故障または誤動作が生じた場合であっても、人身事故、火災事故その他社会的損害等を生じさせないよう、お客様の責任において、冗長設計、延焼対策設計、誤動作防止設計等の安全設計およびエージング処理等、お客様の機器・システムとしての出荷保証を行ってください。特に、マイコンソフトウェアは、単独での検証は困難なため、お客様の機器・システムとしての安全検証をお客様の責任で行ってください。
10. 当社製品の環境適合性等の詳細につきましては、製品個別に必ず当社営業窓口までお問合せください。ご使用に際しては、特定の物質の含有・使用を規制する RoHS 指令等、適用される環境関連法令を十分調査のうえ、かかる法令に適合するようご使用ください。かかる法令を遵守しないことにより生じた損害に関して、当社は、一切その責任を負いません。
11. 当社製品および技術を国内外の法令および規則により製造・使用・販売を禁止されている機器・システムに使用することはできません。当社製品および技術を輸出、販売または移転等する場合は、「外国為替及び外国貿易法」その他日本国および適用される外国の輸出管理関連法規を遵守し、それらの定めるところに従い必要な手続きを行ってください。
12. お客様が当社製品を第三者に転売等される場合には、事前に当該第三者に対して、本ご注意書き記載の諸条件を通知する責任を負うものといたします。
13. 本資料の全部または一部を当社の文書による事前の承諾を得ることなく転載または複製することを禁じます。
14. 本資料に記載されている内容または当社製品についてご不明な点がございましたら、当社の営業担当者までお問合せください。

注 1. 本資料において使用されている「当社」とは、ルネサス エレクトロニクス株式会社およびルネサス エレクトロニクス株式会社が直接的、間接的に支配する会社をいいます。

注 2. 本資料において使用されている「当社製品」とは、注 1 において定義された当社の開発、製造製品をいいます。

(Rev.5.0-1 2020.10)

本社所在地

〒135-0061 東京都江東区豊洲 3-2-24（豊洲フォレシア）

www.renesas.com

お問合せ窓口

弊社の製品や技術、ドキュメントの最新情報、最寄の営業お問合せ窓口に関する情報などは、弊社ウェブサイトをご覧ください。

www.renesas.com/contact/

商標について

ルネサスおよびルネサスロゴはルネサス エレクトロニクス株式会社の商標です。すべての商標および登録商標は、それぞれの所有者に帰属します。

© 2025 Renesas Electronics Corporation. All rights reserved.